Data Scientist

Complete Learning Notes

8 topics | Foundations, worked examples and practice solutions

Scope: The selected career requirements and all transitive prerequisites in the current CareerIQ catalog.

Use this book in prerequisite order. Start with Foundations if a topic is new, then work through the detailed concepts and examples. Experienced learners can focus on tradeoffs, interview questions and applied practice.

These are original AI-assisted study notes, not an accredited textbook or evidence of mastery. Validate software examples against the linked primary documentation and complete the existing assessments or reviewed projects.

Reference books are listed by title and author only. No third-party textbook PDFs are included.

Contents

1. Linear Algebra 3

2. Probability & Statistics 9

3. Python (Advanced) 15

4. NumPy & Pandas Processing 22

5. Advanced SQL & Data Modeling 28

6. Supervised Learning Algorithms 34

7. Model Evaluation & Validation 40

8. Unsupervised & Dimensionality 47

1. Linear Algebra

Prerequisites: No prerequisites in the current competency catalog.

Linear algebra is the mathematical language that underpins many areas of computer science, data analysis, and engineering. It provides tools to represent and manipulate multidimensional data, solve systems of equations, and uncover hidden structure in datasets. Understanding linear algebra enables you to work with transformations, optimize models, and build algorithms that scale. This chapter introduces the core ideas a beginner needs - vectors, matrices, and the operations that combine them - before moving to the three key subtopics: eigenvalues and eigenvectors, singular value decomposition, and tensor operations. By mastering these concepts you will be prepared to tackle machine-learning pipelines, graphics pipelines, and scientific simulations, and you will have a solid foundation for technical interviews that probe your analytical reasoning.

Foundations

A vector is an ordered list of numbers that can be visualized as a point or an arrow in space. Vectors support addition (component-wise) and scalar multiplication, which together define a vector space. A matrix is a rectangular array of numbers that represents a linear transformation from one vector space to another; multiplying a matrix by a vector applies that transformation. Important matrix operations include addition, multiplication, transposition, and inversion. Matrix multiplication is not element-wise; the entry in row i, column j of the product is the dot product of row i of the left matrix with column j of the right matrix. The determinant of a square matrix measures volume scaling and indicates invertibility: a non-zero determinant means the matrix has an inverse. Systems of linear equations can be written as Ax = b, where A is a coefficient matrix, x the unknown vector, and b the result vector. Solving such systems involves Gaussian elimination, LU decomposition, or using the matrix inverse when it exists. Norms quantify vector length (e.g., Euclidean norm) and matrix size (e.g., Frobenius norm). Orthogonality, where vectors have zero dot product, leads to convenient bases that simplify many calculations. These building blocks are essential before exploring eigen-analysis, singular value decomposition, and higher-order tensor algebra.

Eigenvalues & Eigenvectors

An eigenvector of a square matrix A is a non-zero vector v that, when multiplied by A, changes only in magnitude, not direction: A v = λ v. The scalar λ is the eigenvalue associated with v. Eigenpairs reveal intrinsic properties of the transformation, such as scaling factors along principal directions. To find them, solve the characteristic equation det(A - λ I) = 0, where I is the identity matrix; the roots λ are the eigenvalues. For each λ, substitute back into (A - λ I) v = 0 to obtain the eigenvectors, which form the null space of (A - λ I). Eigenvalues can be real or complex, and their multiplicities affect the geometric multiplicity of eigenvectors. In practice, eigen-analysis is used for stability analysis, dimensionality reduction (PCA), and solving differential equations. Numerical methods like the power iteration approximate the dominant eigenpair when exact computation is costly. Trade-offs include sensitivity to rounding errors for ill-conditioned matrices and the need for orthogonal eigenvectors in symmetric matrices.

Worked reasoning: Consider the matrix A = [[4, 1], [2, 3]]. Compute det(A - λ I) = (4-λ)(3-λ) - 2*1 = λ^2 -7λ +10. Solving λ^2 -7λ +10 = 0 gives λ = 5 and λ = 2. For λ = 5, solve (A-5I)v = [[-1,1],[2,-2]]v = 0, yielding eigenvector v1 = [1,1]^T. For λ = 2, solve (A-2I)v = [[2,1],[2,1]]v = 0, giving eigenvector v2 = [-0.5,1]^T (or any scalar multiple). These eigenpairs show that applying A stretches vectors along [1,1] by factor 5 and

along [-0.5,1] by factor 2, illustrating how eigen-analysis decomposes the transformation into independent scaling actions.

Matrix Decomposition (SVD)

Singular value decomposition (SVD) factorizes any m×n matrix M into three components: M = U Σ V^T. U is an m×m orthogonal matrix whose columns are left singular vectors; V is an n×n orthogonal matrix whose columns are right singular vectors; Σ is an m×n diagonal matrix with non-negative singular values σ_i on the diagonal, sorted descending. The singular values are the square roots of the eigenvalues of M^T M (or M M^T). SVD provides optimal low-rank approximations: truncating Σ to its top k singular values yields the best rank-k matrix in the Frobenius norm sense. This property is exploited in data compression, noise reduction, and recommendation systems. Computing SVD involves bidiagonalization followed by iterative QR steps; modern libraries handle this efficiently. Trade-offs include higher computational cost than simpler factorizations (LU, QR) and the need for careful handling of very large sparse matrices, where truncated or randomized SVD variants are preferred.

Worked reasoning: Use M = [[3, 0, 0], [0, 2, 0]]. Then M^T M = diag(9, 4, 0). The two singular values of this 2 by 3 matrix are 3 and 2. A full SVD has U = I_2, V = I_3, and Sigma = M. The rank-1 approximation keeps only 3, giving [[3, 0, 0], [0, 0, 0]]. Its Frobenius error is 2; its squared Frobenius error is 4. Keeping both singular values reconstructs the original exactly, so there is no compression error at rank 2. This example also illustrates that a rectangular matrix has min(m,n) singular values in the usual compact representation, even though M^T M has n eigenvalues.

Tensor Operations

A tensor generalizes scalars (order-0), vectors (order-1), and matrices (order-2) to higher dimensions. An order-k tensor is an array with k indices, e.g., a 3-dimensional array T[i][j][k]. Tensor operations extend matrix algebra: contraction sums over a pair of matching indices, analogous to matrix multiplication; outer product adds new dimensions; and mode-n multiplication multiplies a tensor by a matrix along a specific mode, reshaping that dimension. The Einstein summation convention provides concise notation for these operations. Tensors enable compact representation of multi-way data such as color images (height × width × channel) or word embeddings across contexts. Decompositions like CANDECOMP/PARAFAC (CP) and Tucker factorize tensors into low-rank components, facilitating compression and interpretation. Numerical stability can be challenging because tensor rank is NP-hard to compute, and algorithms may converge to local minima. Trade-offs involve choosing between CP (simpler but potentially less stable) and Tucker (more flexible but higher computational cost).

Worked reasoning: Suppose we have a 2×2×2 tensor T with entries T[0,0,0]=1, T[0,0,1]=2, T[0,1,0]=3, T[0,1,1]=4, T[1,0,0]=5, T[1,0,1]=6, T[1,1,0]=7, T[1,1,1]=8. Contracting over the first and third modes yields a vector C where C[j] = sum_i,k T[i,j,k] δ_{i,k}. Computing C[0] = T[0,0,0]+T[1,0,1]=1+6=7, C[1] = T[0,1,0]+T[1,1,1]=3+8=11. Thus the contraction reduces the tensor to a vector [7,11], illustrating how summing over selected indices extracts aggregated information.

Worked examples

Worked example

```
import numpy as np
A = np.array([[4, 1], [2, 3]])
vals, vecs = np.linalg.eig(A)
print('Eigenvalues:', vals)
print('Eigenvectors:\n', vecs)
```

The code uses NumPy's eig function to compute eigenvalues and eigenvectors of the 2x2 matrix A. The output shows eigenvalues [5. 2.] and corresponding eigenvectors as columns of the matrix printed. Each eigenvector is scaled arbitrarily; any non-zero scalar multiple represents the same direction. This demonstration confirms the manual calculation performed earlier, and illustrates how numerical libraries handle the characteristic polynomial internally. Limitations include sensitivity to rounding errors for matrices with close or complex eigenvalues, and the fact that eig returns complex numbers even for real matrices when necessary.

Applied example: rank-1 reconstruction

```
import numpy as np
M = np.array([[3., 0., 0.], [0., 2., 0.]])
U, s, VT = np.linalg.svd(M, full_matrices=False)
rank1 = U[:, :1] @ np.diag(s[:1]) @ VT[:1, :]
print(rank1)
print(np.linalg.norm(M - rank1, ord='fro'))
```

The rank-1 reconstruction is [[3, 0, 0], [0, 0, 0]], and the Frobenius norm of its error is 2. The squared error is 4, equal to the sum of squares of discarded singular values. Keeping both singular values reconstructs M exactly. Truncation trades fidelity for a lower rank; it is useful for large matrices only when storing the factors actually costs less than storing the original matrix. For a tiny 2 by 3 example, factorization is an illustration rather than a practical storage saving.

Process flow

1. Represent data as vectors or matrices, identifying dimensions and meaning of each axis.
2. Choose appropriate operations: addition for combining, multiplication for applying transformations.
3. Compute eigenvalues and eigenvectors to discover scaling directions of square matrices.
4. Apply SVD to decompose any matrix, inspect singular values to assess rank and energy distribution.
5. If data has more than two modes, organize it as a tensor and perform mode-n multiplications or contractions.
6. Validate results by reconstructing the original structure from decomposed components and measuring error.

Interview and exam checkpoints

Explain the geometric interpretation of eigenvectors and eigenvalues for a 2D linear transformation.

In two dimensions, a linear transformation represented by matrix A maps every vector to a new location. Eigenvectors are special directions that remain on the same line after transformation; they may be stretched or shrunk but not rotated. The eigenvalue associated with an eigenvector quantifies that scaling factor. Geometrically, drawing the eigenvectors shows the axes of the transformation, and the eigenvalues indicate how much the shape expands or contracts along each axis. For example, a matrix that scales x by 3 and y by 2 has eigenvectors aligned with the x- and y-axes and eigenvalues 3 and 2 respectively.

How does truncating singular values in SVD affect data reconstruction?

Truncating singular values means keeping only the largest k singular values and their corresponding singular vectors, discarding the rest. This yields a rank-k approximation that minimizes the Frobenius norm of the reconstruction error. The effect is that most of the variance (energy) captured by the discarded singular values is lost, leading to a smoother or less detailed reconstruction. In image compression, for instance, keeping only a few singular values removes fine details but preserves the overall structure. The trade-off is between compression ratio and fidelity; the squared Frobenius error equals the sum of squares of the omitted singular values.

What are the computational differences between CP and Tucker tensor decompositions?

CP (CANDECOMP/PARAFAC) decomposes a tensor into a sum of rank-one components, requiring the estimation of factor matrices for each mode with a fixed rank. It is simpler and often faster but can suffer from ill-posedness and local minima. Tucker decomposition, on the other hand, factorizes a tensor into a core tensor multiplied by factor matrices along each mode, allowing different ranks per mode. Tucker is more flexible and can capture interactions between components, but it involves computing a potentially large core tensor and is computationally heavier. Algorithms for Tucker typically use higher-order SVD, while CP relies on alternating least squares.

Why is the determinant zero for a matrix with linearly dependent columns, and what does that imply for eigenvalues?

If columns are linearly dependent, the matrix maps some non-zero vector to the zero vector, meaning its transformation collapses volume to zero. The determinant, which measures volume scaling, is therefore zero. A zero determinant implies that zero is an eigenvalue, because there exists a non-zero vector v such that A v = 0·v. This eigenvalue reflects the direction that is annihilated by the transformation, indicating singularity and non-invertibility of the matrix.

Summary

Linear algebra equips you with a systematic way to model and manipulate multidimensional data. Vectors and matrices form the basic objects, supporting operations that represent rotations, scalings, and projections. Eigenvalues and eigenvectors expose intrinsic scaling directions of square transformations, enabling stability analysis and dimensionality reduction. Singular value decomposition extends these ideas to any matrix, providing optimal low-rank approximations and revealing hidden structure through singular values and orthogonal bases. Tensor operations generalize matrix algebra to higher-order data, allowing contractions, mode-n multiplications, and decompositions such as CP and Tucker. Mastery of these concepts gives you the analytical tools to compress data, solve systems efficiently, and design algorithms that scale across domains ranging from machine learning to scientific computing.

Practice questions

1. Find the eigenvalues of matrix B = [[2, -1], [1, 4]].

2. Compute the rank-1 SVD approximation of C = [[3,0,0],[0,2,0]]. Give its Frobenius error.

3. Perform a mode-2 multiplication of a 2×2×2 tensor D with matrix M = [[1,0],[0,2]] and explain the result.

4. Design a data pipeline that uses SVD for image compression. List the steps and justify each.

5. A system of equations is represented by matrix A = [[1,2,3],[2,4,6],[3,6,9]] and vector b = [6,12,18]. Solve for x using the pseudoinverse and discuss the solution space.

6. Compare the trade-offs between using a dense matrix representation versus a sparse representation for a large adjacency matrix in graph algorithms.

Practice solutions

1. Compute the characteristic polynomial det(B - λI) = (2-λ)(4-λ) - (-1)(1) = λ^2 -6λ +9 = 0. Solving λ^2 -6λ +9 = 0 gives λ = (6 ± sqrt(36-36))/2 = 3. Both eigenvalues are λ = 3 (algebraic multiplicity 2).

2. C^T C = diag(9,4,0). The singular values are 3 and 2; choosing the first coordinate directions gives C_1 = [[3,0,0],[0,0,0]]. C-C_1 has a single nonzero entry 2. Thus the Frobenius error is sqrt(4)=2 and its squared error is 4.

3. Using one-based mode numbering, mode 2 acts on the second index. Define Y[i,j,k] = sum_b M[j,b] D[i,b,k]. With M = diag(1,2), entries with j=0 remain unchanged and entries with j=1 double. A slice [[a,b],[c,d]] at fixed k becomes [[a,2b],[c,2d]], equivalently D[:,:,k] @ M.T. The operation scales columns of each slice, not rows.

4. 1) Load grayscale image as matrix I (pixels). 2) Compute SVD: I = U Σ V^T. 3) Choose k based on desired compression ratio (e.g., retain 90% energy). 4) Truncate Σ to top k singular values, forming Σ_k, and keep corresponding columns of U and rows of V^T. 5) Reconstruct compressed image I_k = U_k Σ_k V_k^T. 6) Store U_k, Σ_k, V_k^T instead of full I. Justification: SVD isolates principal components; truncation reduces storage while preserving most visual information. Reconstruction error is minimized in Frobenius norm, ensuring best possible quality for given k.

5. Let u=[1,2,3]^T. A=u*u^T, b=6u and u^T u=14. A has a single nonzero singular value 14. A^+=u*u^T/196. The minimum-norm solution is x_min=(6/14)u=[3/7,6/7,9/7]^T. Check u^T x_min=6, hence A x_min=6u=b. Every solution is x_min+z with u^T z=0. The pseudoinverse returns the unique

minimum-norm solution, not an arbitrary solution.

6. Dense representation stores every entry, using O(n^2) memory, which is wasteful for graphs where edges are far fewer than n^2. Sparse formats (CSR, CSC) store only non-zero entries, reducing memory to O(|E|). Computation on dense matrices can exploit BLAS optimizations and vectorization, yielding fast matrix-vector products, but the overhead is prohibitive for large sparse graphs. Sparse representations enable O(|E|) operations, essential for scalability, but require more complex indexing and may incur indirect memory accesses, reducing cache efficiency. The choice depends on graph density: dense for small, fully connected graphs; sparse for large, real-world networks.

Reference textbooks

Introduction to Linear Algebra - Gilbert Strang

Numerical Linear Algebra - Lloyd N. Trefethen and David Bau III

Primary documentation

https://ocw.mit.edu/courses/18-06-linear-algebra-spring-2010/

Content version: learning-v1 | CareerIQ original AI-assisted educational notes

2. Probability & Statistics

Prerequisites: Linear Algebra

Probability and statistics form the mathematical backbone of data-driven decision making. Whether you are estimating the chance of a coin landing heads, inferring the parameters of a distribution from observed data, or modeling sequential events such as weather patterns, a solid grasp of these ideas is essential. This chapter introduces the core concepts a beginner needs before tackling more sophisticated models. You will learn how to reason about uncertainty with Bayes Theorem, how to fit models to data using Maximum Likelihood Estimation, and how to describe stochastic processes with Markov Chains. By the end, you will be able to translate real-world problems into probabilistic language, perform basic calculations by hand, and write simple Python code to simulate and analyze random phenomena.

Foundations

The study of probability begins with a sample space, the set of all possible outcomes of an experiment. Events are subsets of this space, and a probability measure assigns a number between 0 and 1 to each event, obeying the axioms of non-negativity, normalization, and additivity. Random variables map outcomes to numeric values; their distributions describe how likely each value is. Expectation gives the average outcome, variance quantifies spread, and common families such as the Bernoulli, Binomial, Normal, and Poisson distributions provide templates for modeling. Statistics reverses the direction: given observed data, we aim to infer the underlying distribution or its parameters. Estimators are rules that produce parameter guesses; desirable properties include unbiasedness (average equals true value) and consistency (converges with more data). Hypothesis testing evaluates competing explanations by comparing observed evidence against a null model, often using p-values or confidence intervals. These foundations enable the three subtopics that follow: Bayes Theorem for updating beliefs, Maximum Likelihood Estimation for fitting models, and Markov Chains for describing dependent sequences.

Bayes Theorem

Bayes Theorem provides a systematic way to update the probability of a hypothesis when new evidence arrives. Formally, P(H|E) = P(E|H) * P(H) / P(E), where H is the hypothesis and E is the evidence. The prior P(H) captures belief before seeing data, the likelihood P(E|H) measures how probable the evidence is under the hypothesis, and the posterior P(H|E) is the revised belief. The denominator P(E) acts as a normalizing constant ensuring the posterior sums to one across all hypotheses. Bayes Theorem is powerful because it combines prior knowledge with observed data, allowing iterative refinement as more evidence accumulates. Trade-offs include the need for a well-specified prior, which can be subjective, and computational difficulty when the hypothesis space is large, often requiring approximation methods such as Monte Carlo sampling.

Worked reasoning: Suppose a medical test for a disease is 99% sensitive (true positive) and 95% specific (true negative). The disease prevalence in the population is 1%. Let H be "patient has disease" and E be "test is positive". Prior P(H)=0.01. Likelihood P(E|H)=0.99. False-positive rate is 1-0.95=0.05, so P(E|~H)=0.05. The evidence probability P(E)=P(E|H)P(H)+P(E|~H)P(~H)=0.99*0.01+0.05*0.99=0.0099+0.0495=0.0594. Posterior P(H|E)=0.99*0.01/0.0594≈0.1667. Even with a highly accurate test, the chance the patient actually has the disease after a positive result is only about 16.7% because the disease is rare.

Maximum Likelihood Estimation (MLE)

Maximum Likelihood Estimation selects parameter values that make the observed data most probable under a chosen statistical model. Given data points x1,…,xn assumed independent and identically distributed from a distribution f(x;θ) with parameter θ, the likelihood function L(θ)=∏_{i=1}^n f(x_i;θ) measures joint probability of the data. The MLE θ̂ maximizes L(θ), often more conveniently by maximizing the log-likelihood ℓ(θ)=log L(θ) because products become sums and derivatives are easier. Solving ∂ℓ/∂θ=0 yields candidate estimates; second-order conditions confirm a maximum. MLE is asymptotically unbiased and efficient under regularity conditions, but it can be sensitive to model misspecification and outliers. In cases where the likelihood has multiple peaks, numerical optimization (e.g., gradient ascent) may be required, and care must be taken to avoid local maxima.

Worked reasoning: Assume we observe three independent measurements of a physical quantity that we model as normally distributed with unknown mean μ and known variance σ^2=1. The data are x=[2.1, 1.9, 2.0]. The likelihood is L(μ)=∏_{i=1}^3 (1/√(2π)) exp(-(x_i-μ)^2/2). The log-likelihood ℓ(μ)= -3/2 log(2π) - 0.5∑(x_i-μ)^2. Differentiating: dℓ/dμ = ∑(x_i-μ)=0 □ 3μ = ∑x_i = 2.1+1.9+2.0 = 6.0 □ μ̂ = 2.0. Thus the MLE for the mean is the sample average, a result that holds for any normal distribution with known variance.

Markov Chains

A Markov Chain is a stochastic process that moves between a finite set of states with the property that the next state depends only on the current state, not on the sequence of past states. This memoryless property is formalized as P(S_{t+1}=j | S_t=i, S_{t-1}=k, …) = P(S_{t+1}=j | S_t=i) = T_{ij}, where T is the transition matrix. Each row of T sums to one, representing a probability distribution over next states. Chains can be classified by properties such as irreducibility (every state reachable from any other), aperiodicity (no fixed cycle length), and ergodicity (convergence to a unique stationary distribution π satisfying πT=π). Trade-offs involve model simplicity versus realism: a first-order Markov assumption may be too restrictive for processes with longer memory, prompting higher-order extensions or hidden Markov models.

Worked reasoning: Consider a simple weather model with two states: Sunny (S) and Rainy (R). Transition probabilities are: P(S→S)=0.8, P(S→R)=0.2, P(R→S)=0.4, P(R→R)=0.6. The transition matrix T = [[0.8,0.2],[0.4,0.6]]. Starting from Sunny, the probability of being Sunny after two days is computed by squaring the matrix: T^2 = [[0.8*0.8+0.2*0.4, 0.8*0.2+0.2*0.6],[0.4*0.8+0.6*0.4, 0.4*0.2+0.6*0.6]] = [[0.72,0.28],[0.56,0.44]]. Thus P(S after two days | start S)=0.72. The stationary distribution solves π = πT, yielding π = [0.667, 0.333]; in the long run, about two-thirds of days are Sunny.

Worked examples

Worked example

```
import random

def simulate_coin(p_head, n):
    heads = sum(1 for _ in range(n) if random.random() < p_head)
    return heads / n

# Estimate probability of heads for a biased coin
est = simulate_coin(0.7, 10000)
print('Estimated P(heads):', round(est, 4))
```

The code defines a simple Monte Carlo simulation for a biased coin with true head probability 0.7. By drawing 10,000 random numbers and counting how many fall below 0.7, we approximate the empirical

frequency of heads. The printed estimate will be close to 0.7, illustrating the law of large numbers: as the number of trials grows, the sample proportion converges to the true probability. Limitations include random variance; a single run may deviate by a few percent, and the method becomes inefficient for extremely rare events where many more trials are needed to obtain stable estimates.

Applied example

```
# Pseudocode for MLE of exponential rate
# data: list of observed lifetimes x_i
sum_x = sum(data)
N = len(data)
# Log-likelihood derivative set to zero gives lambda_hat = N / sum_x
lambda_hat = N / sum_x
print('MLE rate λ =', lambda_hat)
```

The pseudocode demonstrates the derivation of the Maximum Likelihood Estimator for the rate parameter λ of an exponential distribution, f(x;λ)=λ e^{-λx} for x≥0. The log-likelihood is ℓ(λ)=N log λ - λ∑x_i. Differentiating with respect to λ and setting the derivative to zero yields N/λ - ∑x_i = 0, which solves to λ̂ = N / ∑x_i, the reciprocal of the sample mean. For independent exponential observations and N>1, this estimator is biased upward: E[lambda_hat]=N*lambda/(N-1). It is consistent and asymptotically unbiased, and easy to compute, making it a common choice in reliability engineering where lifetimes follow an exponential decay.

Process flow

1. Define the random experiment and enumerate all possible outcomes (sample space).
2. Assign probabilities to elementary events ensuring the axioms of probability hold.
3. Introduce random variables and derive their probability mass or density functions.
4. Apply Bayes Theorem to update beliefs when new evidence is observed.
5. Formulate a likelihood function for the observed data and solve for the MLE.
6. Construct a transition matrix if the problem involves sequential dependence, then analyze its properties (stationary distribution, mixing time).
7. Validate results with simulation or analytical checks, and interpret the outcomes in the context of the original problem.

Interview and exam checkpoints

Explain the difference between a prior and a likelihood in Bayesian inference.

A prior represents the probability distribution of a hypothesis before seeing any data; it encodes existing knowledge or assumptions. The likelihood, on the other hand, quantifies how probable the observed data are under each possible hypothesis. In Bayes Theorem, the prior is multiplied by the likelihood to produce the unnormalized posterior, which is then normalized by the evidence. The prior influences the posterior especially when data are scarce, while the likelihood dominates as more data accumulate.

Derive the MLE for the parameter p of a Bernoulli distribution given n observations.

For a Bernoulli trial, the probability mass function is f(x;p)=p^x (1-p)^{1-x} for x□{0,1}. Given observations x1,…,xn, the likelihood is L(p)=∏_{i=1}^n p^{x_i}(1-p)^{1-x_i}=p^{∑x_i}(1-p)^{n-∑x_i}. The log-likelihood is ℓ(p)=∑x_i log p + (n-∑x_i) log(1-p). Differentiating with respect to p and setting to zero: (∑x_i)/p - (n-∑x_i)/(1-p)=0. Solving gives p̂ = (∑x_i)/n, which is simply the sample mean of the binary outcomes.

What conditions guarantee that a Markov chain has a unique stationary distribution?

A finite Markov chain that is both irreducible (every state can be reached from any other) and aperiodic (the chain does not get trapped in cycles of fixed length) is guaranteed to have a unique stationary distribution. Irreducibility ensures the chain is one communicating class, while aperiodicity prevents oscillations. Under these conditions, the chain is ergodic, meaning that regardless of the initial state, the distribution of states converges to the stationary distribution as time goes to infinity.

How would you handle a situation where the likelihood function is intractable for Bayesian inference?

When the likelihood cannot be evaluated analytically, we resort to approximation techniques. One common approach is Markov Chain Monte Carlo (MCMC), which constructs a Markov chain whose stationary distribution matches the posterior, allowing us to draw samples without explicit likelihood values. Another method is Approximate Bayesian Computation (ABC), which simulates data from the model and accepts parameter values that produce simulated data close to the observed data according to a distance metric. Both methods trade computational cost for the ability to perform inference in complex models.

Summary

Probability and statistics provide a language for quantifying uncertainty and extracting insight from data. Starting from basic probability axioms, we learned to model random variables, compute expectations, and assess variability. Bayes Theorem equips us to revise beliefs in light of new evidence, balancing prior knowledge with observed data. Maximum Likelihood Estimation offers a principled way to fit parametric models by maximizing the probability of the observed sample, yielding estimators that are often simple functions of the data. Markov Chains extend probability to sequences where the future depends only on the present, enabling analysis of stochastic processes through transition matrices, stationary distributions, and convergence properties. Together, these tools form a foundation for more advanced topics such as Bayesian hierarchical models, hidden Markov models, and modern machine-learning algorithms.

Practice questions

1. A biased die has probability 0.2 of landing on 6 and equal probability for the other faces. What is the probability of rolling a 6 exactly twice in three independent throws?

2. Given data points [5.2, 4.8, 5.0] assumed drawn from N(μ, σ^2) with known σ=0.1, compute the MLE for μ.

3. In a two-state Markov chain with transition matrix [[0.9,0.1],[0.3,0.7]], compute the probability of being in state 2 after two steps starting from state 1.

4. Write a Python function that computes the posterior probability of a hypothesis H given binary evidence E using Bayes Theorem. Use inputs prior, likelihood, and evidence probability.

5. Design a simple weather prediction system using a first-order Markov chain. Discuss the trade-offs of using only the previous day's weather versus incorporating a longer history.

6. A sample of 50 measurements has mean 12.3 and standard deviation 2.1. Assuming normality, construct a 95% confidence interval for the true mean.

Practice solutions

1. Let p=0.2 for a 6, q=0.8 for not-6. The number of successes follows a Binomial(3,0.2). P(X=2)=C(3,2)*p^2*q = 3 * 0.04 * 0.8 = 0.096.

2. The MLE for μ is the sample mean: (5.2+4.8+5.0)/3 = 5.0. Thus μ̂ = 5.0.

3. Square the matrix: T^2 = [[0.9*0.9+0.1*0.3, 0.9*0.1+0.1*0.7],[0.3*0.9+0.7*0.3, 0.3*0.1+0.7*0.7]] = [[0.84,0.16],[0.48,0.52]]. Starting from state 1, the probability of being in state 2 after two steps is the (1,2) entry = 0.16.

4.
```
def bayes_posterior(prior, likelihood, evidence):
    return (likelihood * prior) / evidence
# Example usage:
# prior=0.01, likelihood=0.99, evidence=0.0594
# posterior ≈ 0.1667
```

5. Model states as {Sunny, Cloudy, Rainy}. Estimate transition probabilities from historical data, forming a 3×3 matrix. The system predicts tomorrow's weather by multiplying the current state vector by the matrix. Trade-offs: a first-order model is easy to train and requires little data, but it cannot capture patterns like a

multi-day front that influence weather beyond one day. Extending to a second-order chain (conditioning on the last two days) improves accuracy for such patterns but increases the number of parameters (from 9 to 27 probability entries, or 6 to 18 free parameters after row-sum constraints) and demands more data to estimate them reliably, raising the risk of overfitting.

6. For n=50, use t_{0.025,49}≈2.01. Margin = t * s/√n = 2.01 * 2.1 / √50 ≈ 2.01 * 0.297 = 0.597. CI = 12.3 ± 0.597 → (11.703, 12.897).

Reference textbooks

Introduction to Probability - Joseph K. Blitzstein and Jessica Hwang

All of Statistics - Larry Wasserman

Primary documentation

https://seeing-theory.brown.edu/

Content version: learning-v1 | CareerIQ original AI-assisted educational notes

3. Python (Advanced)

Prerequisites: No prerequisites in the current competency catalog.

Python has become the lingua franca for many advanced software tasks, from high-performance networking to low-level system integration. Mastering the advanced features of the language - concurrency with AsyncIO, the power of metaclasses and decorators, and the ability to profile memory or extend Python with C - enables developers to write code that is both expressive and efficient. This chapter introduces these capabilities in a self-contained way, starting from the core ideas and building toward practical implementations. By the end, a beginner will understand when to use asynchronous constructs, how to shape class creation dynamically, and how to measure and improve memory usage, preparing them for real-world projects and technical interviews.

Foundations

AsyncIO is Python's built-in library for cooperative multitasking. Unlike pre-emptive threads, coroutines voluntarily yield control, allowing a single thread to manage many I/O-bound tasks without the overhead of context switches. The event loop schedules these coroutines, and futures represent pending results. Metaclasses are the "class of a class"; they intercept class creation, enabling automatic attribute injection or validation. Decorators are higher-order functions that wrap other callables, adding behavior without modifying the original code. Together they support clean, reusable patterns. Memory profiling in Python involves tools such as tracemalloc and objgraph, which track allocation sites and object graphs. When Python's performance limits are reached, C-extensions let developers write critical sections in C, exposing functions to Python via the CPython API. Understanding reference counting, the Global Interpreter Lock (GIL), and the distinction between pure Python objects and native buffers is essential before venturing into extensions. These foundations form a toolkit for writing scalable, maintainable, and high-performance Python applications.

AsyncIO

AsyncIO provides an event-driven framework where coroutines, defined with async def, can pause at await points. The await keyword yields control back to the loop, which then runs other ready coroutines. This model excels for I/O-heavy workloads such as web servers, database clients, or file processing, because the thread never blocks on network or disk latency. Trade-offs include added complexity in error handling and debugging, as stack traces become fragmented across await boundaries. Moreover, CPU-bound work still blocks the loop unless offloaded to a thread or process pool.

Worked reasoning: Consider a simple HTTP fetcher that retrieves three URLs concurrently. The coroutine fetch(url) opens an aiohttp.ClientSession, awaits response.read(), and returns the length. The main coroutine gathers the three fetch calls with asyncio.gather, allowing the event loop to interleave network waits. The total runtime approximates the longest single request, not the sum, demonstrating true concurrency without threads.

Metaclasses & Decorators

Metaclasses intervene during class creation by overriding __new__ or __init__ in a subclass of type. They can inject methods, enforce naming conventions, or register classes in a registry automatically. Decorators, on the other hand, are applied to functions or methods at definition time, returning a wrapped callable that can log, cache, or modify arguments. Using both together yields powerful patterns: a metaclass can apply a decorator to every method of a class, ensuring uniform behavior such as timing or access control. The main trade-off is readability; excessive metaprogramming can obscure intent, making maintenance harder for newcomers.

Worked reasoning: A metaclass AutoTimer scans a class dictionary, wrapping each callable with a timing_decorator that records start and end timestamps. When an instance method is called, the decorator prints the elapsed time. Defining class MyWorker(metaclass=AutoTimer) automatically equips all its methods with timing without manually decorating each one.

Memory Profiling & C-Extensions

Memory profiling starts with tracemalloc.start() to capture allocation snapshots. By comparing snapshots, developers can pinpoint functions that allocate the most bytes. objgraph.show_refs can visualize reference cycles that prevent garbage collection. When profiling reveals hot spots, a C-extension can replace a Python loop with a compiled routine, reducing per-iteration overhead and memory churn. The CPython API requires careful reference counting: Py_INCREF and Py_DECREF manage object lifetimes, and the GIL must be held when manipulating Python objects. The trade-off is development effort and portability; C code must be compiled for each target platform, and bugs can crash the interpreter.

Worked reasoning: A pure-Python function sum_range(n) computes the sum of squares using a for loop, allocating a new integer each iteration. Profiling shows O(n) allocations. Rewriting the core loop in C as sum_range_c(PyObject *self, PyObject *args) uses a single PyLong_FromLong result, dramatically lowering allocation count and memory usage. The extension is built with setuptools' Extension module and imported as import sum_ext.

Worked examples

Worked example

```
import asyncio
import aiohttp

async def fetch(url):
    async with aiohttp.ClientSession() as s:
        async with s.get(url) as r:
            data = await r.read()
            return len(data)

async def main():
    urls = ['https://example.com', 'https://httpbin.org/ip',
    'https://httpbin.org/uuid']
    lengths = await asyncio.gather(*(fetch(u) for u in urls))
    print('Lengths:', lengths)

asyncio.run(main())
```

The script defines fetch as an async coroutine that opens a client session, performs a GET request, and awaits the response body. main builds a list of three URLs and uses asyncio.gather to schedule all fetch

calls concurrently. Because each await yields while the network is idle, the total runtime is close to the slowest request rather than the sum of three round-trips. Limitations include the need for an event loop (cannot be called from a blocking context without run_until_complete) and that CPU-bound work inside fetch would still block the loop.

Applied example

```
# Pseudocode for a metaclass that decorates all methods
class AutoLog(type):
    def __new__(mcls, name, bases, dct):
        def log_wrapper(func):
            def inner(*args, **kw):
                print(f'Calling {func.__name__}')
                return func(*args, **kw)
            return inner
        for attr, val in dct.items():
            if callable(val):
                dct[attr] = log_wrapper(val)
        return super().__new__(mcls, name, bases, dct)

class Service(metaclass=AutoLog):
    def start(self):
        print('started')
    def stop(self):
        print('stopped')

svc = Service()
svc.start()
svc.stop()
```

AutoLog is a metaclass that iterates over the class dictionary during creation. For each callable attribute, it replaces the original function with a wrapped version that prints a log line before invoking the original. When Service is defined, both start and stop are automatically logged without explicit @decorator usage. This pattern is useful for cross-cutting concerns like logging, authentication, or timing. The trade-off is that every method, including private helpers, receives the wrapper, which may add overhead or interfere with introspection tools.

Process flow

1. 1. Identify a performance or architectural problem (e.g., many concurrent I/O calls).
2. 2. Choose AsyncIO if the workload is I/O-bound; rewrite blocking functions as async coroutines.
3. 3. Insert await points at each I/O operation and run coroutines via asyncio.run or an existing loop.
4. 4. If repetitive cross-cutting behavior is needed, design a decorator or metaclass to apply it uniformly.
5. 5. Profile memory with tracemalloc.start(); capture snapshots before and after the critical section.
6. 6. Analyze allocation differences; if Python overhead is high, implement the hot loop as a C-extension.
7. 7. Test the integrated solution, verify correctness, and measure speed and memory improvements.

Interview and exam checkpoints

Explain how AsyncIO differs from threading and when you would prefer one over the other.

AsyncIO uses a single thread with an event loop that schedules coroutines, allowing many I/O operations to progress without OS thread switches. Threading creates multiple OS threads, each with its own stack and context, incurring higher memory usage and synchronization costs. Prefer AsyncIO for high-concurrency I/O-bound tasks like web servers, where the GIL is not a bottleneck. Use threading when you need true parallelism for CPU-bound work or when interacting with libraries that block the interpreter and cannot be made async.

What is a metaclass in Python and how does it differ from a class decorator?

A metaclass is a subclass of type that controls the creation of classes; its __new__ or __init__ runs when the class statement is executed, allowing injection or modification of attributes before the class object exists. A class decorator is a function that receives a class object after it has been created and returns a (possibly modified) class. Metaclasses operate at class definition time, affecting the class's structure, while decorators act on the finished class, typically adding behavior. Metaclasses are more powerful but also more complex and less commonly needed.

How does tracemalloc help you find memory leaks, and what are its limitations?

tracemalloc records the call stack of each memory allocation, enabling you to take snapshots and compare them to see which lines allocate the most bytes. By filtering snapshots you can locate unexpected growth in specific functions, indicating a leak. Limitations include that it only tracks Python-level allocations, not memory allocated by native extensions, and it adds overhead (typically 5-10%). It also cannot detect reference cycles that keep objects alive unless you inspect the object graph with additional tools.

Describe the steps to safely expose a C function to Python using the CPython API.

First, write a C function that follows the PyCFunction signature, accepting PyObject *self and PyObject *args. Parse arguments with PyArg_ParseTuple, perform the computation, and return a new reference using PyLong_FromLong or similar. Next, create a method table (PyMethodDef array) listing the function name, pointer, flags, and docstring. Define a module initialization function (PyMODINIT_FUNC PyInit_modname) that calls PyModule_Create with a PyModuleDef struct containing the method table. Finally, compile the source into a shared library using setuptools' Extension module, and import it in Python. Throughout, manage reference counts with Py_INCREF/Py_DECREF and hold the GIL when interacting with Python objects.

Summary

Advanced Python equips developers with tools to handle massive concurrency, dynamic class behavior, and low-level performance tuning. AsyncIO replaces threads for I/O-bound workloads by using cooperative coroutines and an event loop, yielding efficient scaling with minimal overhead. Metaclasses and decorators enable systematic injection of cross-cutting concerns, allowing clean, reusable modifications to classes and functions. Memory profiling with tracemalloc and objgraph reveals hidden allocation patterns, guiding decisions to rewrite hot paths as C-extensions when Python's interpreter overhead becomes a bottleneck. Together, these techniques form a cohesive strategy: identify bottlenecks, apply the appropriate abstraction, measure impact, and iterate. Mastery of these concepts prepares engineers for real-world systems where responsiveness, resource usage, and maintainability are paramount.

Practice questions

1. Write an async function that reads three files concurrently and returns their total line count. Use only standard-library asyncio and the third-party aiofiles package.

2. Create a metaclass that automatically registers every subclass in a global registry dictionary keyed by class name.

3. Using tracemalloc, compare memory usage of a list comprehension versus a generator expression that produces the squares of numbers 1..1000. Report the difference in bytes.

4. Design a small C-extension that provides a function fast_sum(arr) returning the sum of a Python list of integers. Outline the steps and explain reference-count handling.

5. Explain a scenario where using a decorator for caching (memoization) could degrade performance instead of improving it.

6. You have a web service that must handle 10,000 simultaneous connections, each performing short HTTP GETs. Choose between AsyncIO and a thread pool, justify your choice, and sketch the high-level code structure.

Practice solutions

1. import asyncio, aiofiles

```
async def count_lines(path):
    async with aiofiles.open(path, 'r') as f:
        cnt = 0
        async for _ in f:
            cnt += 1
        return cnt

async def total():
    files = ['a.txt', 'b.txt', 'c.txt']
    results = await asyncio.gather(*(count_lines(p) for p in files))
    return sum(results)

print(asyncio.run(total()))
# aiofiles delegates ordinary file operations to worker threads, counts lines asynchronously, and aggregates the counts. Because file I/O yields control, the three reads overlap, potentially overlapping
```

waits; speedup depends on the filesystem and workload.

2. registry = {}

```
class RegisterMeta(type):
    def __new__(mcls, name, bases, dct):
        cls = super().__new__(mcls, name, bases, dct)
        registry[name] = cls
        return cls

class Base(metaclass=RegisterMeta):
    pass

class Foo(Base):
    pass

class Bar(Base):
    pass

print(registry) # {'Base': <class '__main__.Base'>, 'Foo': <class '__main__.Foo'>, 'Bar': <class
'__main__.Bar'>}
# Each time a new subclass is defined, RegisterMeta inserts it into the registry, enabling factory patterns or
plugin discovery without manual bookkeeping.
```

3. import gc, tracemalloc

```
def measure(factory):
    gc.collect()
    tracemalloc.start()
    value = factory() # retain the object while measuring
    current, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return current, peak

list_bytes = measure(lambda: [i*i for i in range(1, 1001)])
generator_bytes = measure(lambda: (i*i for i in range(1, 1001)))
print("list current/peak:", list_bytes)
print("generator current/peak:", generator_bytes)
print("current difference:", list_bytes[0] - generator_bytes[0])
```

Results depend on the Python build and allocator; do not hardcode a byte count. The list retains computed elements. The generator retains its suspended state and produces values when consumed. This measures construction, not the work or memory of consuming and retaining all generated results. Discarding both objects before snapshots would not measure their retained memory.

4. Define a method table and module initialization function, then expose a callable that parses an iterable argument. Obtain an iterator with PyObject_GetIter and release its new reference on every exit path. Each non-null PyIter_Next result is also a new reference: process it, then decrement it. A null iteration result can mean exhaustion or an exception; check PyErr_Occurred after the loop. For arbitrary Python integers, maintain a Python integer accumulator using checked PyNumber_Add results and correct reference ownership. A C long accumulator can overflow even when each input fits individually; do not use unchecked signed addition. Return a new reference to the result on success and NULL with an exception

set on failure. Include Python.h, define the method table/module entry point and use an appropriate extension build configuration. Test empty input, non-integers, huge integers, an iterator that raises, and repeated calls under leak checking. A C extension is not automatically faster; benchmark against Python's built-in sum before accepting its maintenance cost.

5. If the decorated function is called with a large number of unique arguments, the cache grows without reuse, consuming memory and adding lookup overhead for each call. For example, a function that processes streaming sensor data with ever-changing timestamps will store a new entry per call, eventually causing cache thrashing and increased GC pressure. In such cases, the decorator's dictionary look-ups and memory usage outweigh any benefit, and a simple direct call is faster.

6. AsyncIO is the better fit because the workload is I/O-bound and each request spends most of its time waiting for network responses. Creating 10,000 OS threads would exhaust system resources (stack memory, scheduler overhead). With AsyncIO, a single thread runs an event loop that schedules all coroutines, keeping memory usage low. High-level code:

```
import asyncio, aiohttp

async def fetch(session, url):
    async with session.get(url) as resp:
        return await resp.text()

async def main(urls):
    async with aiohttp.ClientSession() as sess:
        tasks = [fetch(sess, u) for u in urls]
        results = await asyncio.gather(*tasks)
        return results

urls = ['http://service/api'] * 10000
print(asyncio.run(main(urls)))
# The event loop multiplexes the 10,000 coroutines, achieving high concurrency with minimal threads.
```

Reference textbooks

Fluent Python - Luciano Ramalho

Effective Python - Brett Slatkin

Primary documentation

https://docs.python.org/3/tutorial/

Content version: learning-v1 | CareerIQ original AI-assisted educational notes

4. NumPy & Pandas Processing

Prerequisites: Python (Advanced)

Data scientists spend most of their time cleaning, reshaping, and aggregating data before any model can be built. NumPy provides the low-level, fast array engine that underpins most scientific Python code, while Pandas builds a higher-level, tabular abstraction that makes time-series handling, grouping, and missing-value logic intuitive. Mastering these libraries enables you to write code that runs at C-speed, avoid Python loops, and express complex data pipelines in a few lines. This chapter introduces the core ideas of vectorized computation, memory-efficient storage, and time-series alignment, then shows how they combine in real-world workflows. By the end you will be able to read large files, transform them without copying data unnecessarily, and align disparate series for analysis, all while keeping an eye on performance and correctness.

Foundations

NumPy arrays are homogeneous, n-dimensional containers of fixed-size items stored in a contiguous block of memory. Because the data type is known, the interpreter can call optimized C loops for arithmetic, eliminating the overhead of Python's dynamic typing. Vectorization means applying an operation to an entire array at once; broadcasting extends this by allowing arrays of different shapes to interact as long as their dimensions are compatible, following the rule that dimensions of size one are virtually repeated. Pandas introduces the DataFrame, a collection of labeled Series objects, each of which can hold a different dtype. The index provides alignment semantics: when two Series are combined, Pandas automatically matches rows by label, inserting NaN where data is missing. Memory mapping (numpy.memmap) lets you treat a file on disk as an array, loading only the needed chunks into RAM, which is essential for datasets larger than available memory. Time-series alignment builds on the index concept, using timestamps as labels and providing resampling, forward-fill, and interpolation utilities. Understanding these foundations - contiguous storage, dtype uniformity, label-based alignment, and lazy loading - prepares you to write code that is both fast and expressive.

Vectorization & Broadcasting

Vectorization replaces explicit Python loops with array-wide operations, leveraging SIMD instructions and cache-friendly memory access. The key trade-off is that you must think in whole-array terms; element-wise logic must be expressed with boolean masks or where functions. Broadcasting allows operations between arrays of different shapes by virtually replicating the smaller array along dimensions of size one. This avoids costly data copies but requires careful shape planning to prevent unintended broadcasting that yields wrong results. Understanding the broadcasting rules - right-most dimensions align first, and a dimension of size one can be stretched - helps you design compact, error-free code.

Worked reasoning: Suppose we have a 3x4 matrix A and a length-4 row vector v. A = [[1,2,3,4],[5,6,7,8],[9,10,11,12]] and v = [1,0,1,0]. Adding v to A triggers broadcasting: v is treated as [[1,0,1,0]] repeated across three rows, producing [[2,2,4,4],[6,6,8,8],[10,10,12,12]]. No extra memory is allocated for the repeated v; NumPy computes each sum on the fly.

Memory Mapping

Memory mapping creates an array interface to a binary file without loading the entire file into RAM. numpy.memmap opens the file, sets the dtype and shape, and maps the file's bytes into the process's address space. Reads and writes operate directly on the file, so changes persist without an explicit save step. The main trade-off is that random access can be slower than in-memory arrays due to page faults, and the file must remain unchanged in size and layout while the memmap is active. It is ideal for large, read-only datasets or for incremental updates where only a slice is needed at a time.

Worked reasoning: Imagine a binary file 'data.bin' containing 10,000 int32 values. Using memmap: import numpy as np; mm = np.memmap('data.bin', dtype='int32', mode='r', shape=(10000,)). Accessing mm[5000] reads the 5,001st integer directly from disk. If you compute mm[::2] + 5, NumPy reads only the even-indexed elements, adds 5, and returns a new in-memory array, leaving the original file untouched.

Time Series Alignment

Pandas aligns series by their index labels, which are often timestamps. When two series with different sampling frequencies are combined, Pandas reindexes them to a common timeline, inserting NaN where data is missing. You can then fill gaps using forward fill, backward fill, or interpolation. The trade-off lies in choosing an appropriate frequency: upsampling introduces many NaNs that must be filled, while downsampling discards information. Resampling with .resample() aggregates data (e.g., taking the mean per hour), while .asfreq() simply changes the index without aggregation, preserving original values where possible.

Worked reasoning: Consider s1 = pd.Series([10,12,14], index=pd.to_datetime(['2021-01-01','2021-01-02','2021-01-04'])) and s2 = pd.Series([100,101], index=pd.to_datetime(['2021-01-01','2021-01-03'])). Adding s1 + s2 yields a series indexed on the union of dates: 2021-01-01 -> 110, 2021-01-02 -> NaN, 2021-01-03 -> NaN, 2021-01-04 -> NaN. Using s2.reindex(s1.index, method='ffill') fills missing dates with the last known value, enabling meaningful addition.

Worked examples

Worked example

```
import numpy as np
A = np.arange(1,13).reshape(3,4)
v = np.array([1,0,1,0])
B = A + v  # broadcasting
print(B)
```

The code creates a 3x4 matrix A with values 1 through 12 and a row vector v of length four. Adding v to A triggers broadcasting: v is virtually repeated for each row, so each element of A is increased by the corresponding element of v. The printed result is [[2 2 4 4]
 [6 6 8 8]
 [10 10 12 12]]. This demonstrates how NumPy can perform element-wise addition without explicit loops, saving both code length and execution time. The operation does not allocate a separate copy of v for each row; the broadcast is handled internally, which is why memory usage stays low.

Applied example

```
import pandas as pd
import numpy as np
# create two time series with different frequencies
idx1 = pd.date_range('2022-01-01', periods=5, freq='D')
idx2 = pd.date_range('2022-01-01', periods=3, freq='2D')
s1 = pd.Series(np.arange(5), index=idx1)
s2 = pd.Series(np.arange(3)*10, index=idx2)
# align and forward-fill missing values
s2_aligned = s2.reindex(s1.index, method='ffill')
result = s1 + s2_aligned
print(result)
```

Here we build two Series: s1 has daily data 0-4, while s2 has data every two days (0,10,20). Reindexing s2 to s1's daily index with method='ffill' propagates the last known value forward, so s2_aligned becomes [0,0,10,10,20]. Adding the aligned series yields [0,1,12,13,24]. This example shows how Pandas automatically aligns on timestamps and how forward filling resolves gaps, a common requirement when merging sensor readings with different sampling rates. The limitation is that forward fill assumes the last observation remains valid, which may not hold for rapidly changing signals.

Process flow

1. 1. Load raw data into a NumPy array or Pandas DataFrame, specifying dtype to avoid implicit conversion.
2. 2. If the file is larger than memory, replace np.load with np.memmap to map the file on disk.
3. 3. Apply vectorized transformations (e.g., scaling, masking) using NumPy broadcasting to avoid loops.
4. 4. Convert the cleaned array to a DataFrame, assigning meaningful column names and a datetime index.
5. 5. Align multiple time-series using the index; use reindex with forward/backward fill or interpolation as needed.
6. 6. Perform group-by or resample operations to aggregate data to the desired frequency.
7. 7. Export the final tidy DataFrame to CSV or a binary format, ensuring that no intermediate copies remain in memory.

Interview and exam checkpoints

Explain how broadcasting works in NumPy and give a case where it can lead to unexpected results.

Broadcasting allows arrays of different shapes to participate in arithmetic by virtually replicating the smaller array along dimensions of size one. The rule is that dimensions are compared from the right; if they are equal or one of them is one, broadcasting proceeds. An unexpected result occurs when a programmer assumes a (3,1) array will broadcast across columns, but actually it broadcasts across rows, leading to shape (3, n) instead of (n, 3). This can produce transposed results or silent shape mismatches if not checked.

What are the advantages and disadvantages of using numpy.memmap for large datasets?

Advantages include low memory footprint, as only the accessed portions are loaded, and the ability to modify data on disk without an explicit save step. Disadvantages are slower random access due to page faults, the requirement that the file layout (dtype, shape) remain constant, and limited support for complex indexing operations that may trigger many small reads, degrading performance.

How does Pandas handle arithmetic between two Series with non-matching timestamps?

Pandas aligns the Series on the union of their indices. For timestamps present in only one Series, the result contains NaN for the missing operand. The user can then fill these NaNs using methods like fillna, forward fill, or interpolation, depending on the semantics of the data. This alignment ensures that operations are label-driven rather than position-driven.

Describe a scenario where you would prefer to use DataFrame.resample() over DataFrame.asfreq().

When aggregating high-frequency sensor data to a lower frequency (e.g., minute to hourly), you need to compute a summary statistic such as the mean or sum for each hour. resample() allows you to specify the aggregation function, whereas asfreq() merely changes the index without aggregating, potentially leaving multiple rows per new period and requiring manual reduction.

Summary

NumPy provides the fast, memory-contiguous array core that enables vectorized arithmetic and broadcasting, eliminating Python loops. Pandas builds on this foundation with labeled Series and DataFrames, offering automatic alignment of data by index, powerful time-series utilities, and group-by aggregation. Memory mapping extends NumPy to handle datasets larger than RAM by treating files as arrays, while careful use of broadcasting avoids unnecessary copies. Together, these tools let you ingest raw data, transform it efficiently, align disparate time series, and produce tidy outputs ready for analysis or modeling. Mastery of these concepts reduces runtime, prevents subtle bugs from misaligned data, and prepares you for real-world data pipelines.

Practice questions

1. Given A = np.array([[2,4],[6,8]]) and B = np.array([1,2]), compute A - B using broadcasting and show the result.

2. Create a memmap of a file 'ints.bin' containing 100 int16 numbers from 0 to 99. Then compute the sum of the first 10 elements without loading the whole file.

3. You have two daily price series: s1 indexed Jan 1-5 with values [100,101,102,103,104]; s2 indexed Jan 3-7 with values [200,201,202,203,204]. Align them, forward-fill missing values, and compute the daily spread s1 - s2.

4. Design a data ingestion pipeline that reads a 2 GB CSV, filters rows where column 'status' == 'active', and writes the result to a compressed Parquet file, using vectorized operations and minimal memory.

5. Explain the trade-off between forward-fill and linear interpolation when aligning irregular time-series data.

6. Write a NumPy function that takes a 2-D integer array and returns a new array where each row is normalized to have zero mean and unit variance, using only vectorized operations.

Practice solutions

1. A - B triggers broadcasting of B to shape (2,2): B becomes [[1,2],[1,2]]. Subtraction yields [[1,2],[5,6]]. Code: import numpy as np; A=np.array([[2,4],[6,8]]); B=np.array([1,2]); print(A-B) -> [[1 2]
 [5 6]].

2. import numpy as np; mm = np.memmap('ints.bin', dtype='int16', mode='r', shape=(100,)); total = mm[:10].sum(); print(total) -> sum of 0..9 = 45. The memmap reads only the first 10 values from disk.

3. import pandas as pd; idx1=pd.date_range('2022-01-01', periods=5); idx2=pd.date_range('2022-01-03', periods=5); s1=pd.Series([100,101,102,103,104], index=idx1); s2=pd.Series([200,201,202,203,204], index=idx2); s2_aligned=s2.reindex(s1.index, method='ffill'); spread=s1 - s2_aligned; Result: 2022-01-01 NaN, 2022-01-02 NaN, 2022-01-03 -100, 2022-01-04 -102, 2022-01-05 -104.

4. Use pandas.read_csv with chunksize to stream the file, e.g., for chunk in pd.read_csv('big.csv', chunksize=100000, dtype={'status':'category'}): filtered = chunk[chunk['status']=='active']; filtered.to_parquet('out.parquet', engine='pyarrow', compression='snappy', index=False, append=True). This avoids loading the whole file, leverages vectorized boolean indexing, and writes incrementally to a compressed columnar format.

5. Forward-fill assumes the last observed value remains valid until a new measurement arrives, which is simple and fast but can introduce step-like artifacts, especially for rapidly changing signals. Linear interpolation estimates intermediate values by assuming a straight line between known points, providing smoother estimates but requiring at least two surrounding observations and potentially misrepresenting non-linear dynamics. Choose forward-fill for categorical or slowly varying data, interpolation for continuous measurements where smoothness is desired.

6. def normalize_rows(arr):
    mean = arr.mean(axis=1, keepdims=True)
    std = arr.std(axis=1, keepdims=True)
    return (arr - mean) / std

```
# Example: A=np.array([[1,2,3],[4,5,6]]); print(normalize_rows(A))
# Output: [[-1.2247 0. 1.2247]
# [-1.2247 0. 1.2247]]
```

Reference textbooks

Python for Data Analysis - Wes McKinney

Python Data Science Handbook - Jake VanderPlas

Primary documentation

https://pandas.pydata.org/docs/user_guide/index.html

Content version: learning-v1 | CareerIQ original AI-assisted educational notes

5. Advanced SQL & Data Modeling

Prerequisites: Python (Advanced)

Advanced SQL and data modeling are essential for turning raw data into actionable insight in modern applications. While basic SELECT statements retrieve rows, advanced techniques let you compute running totals, rank results, traverse hierarchical data, and understand how the database engine turns a query into a plan. Mastering these tools enables you to write queries that are both expressive and performant, a skill highly prized in data-intensive roles. This chapter introduces the core concepts of window functions, recursive common table expressions, and query execution plans, building a solid foundation before you tackle optimization and complex modeling tasks.

Foundations

SQL is a declarative language: you describe *what* result you need, not *how* to compute it. The database optimizer interprets the statement, rewrites it, and chooses an execution strategy based on statistics, indexes, and cost models. Understanding this process is crucial because advanced features such as window functions and recursive CTEs rely on the optimizer to allocate resources efficiently. A window function computes a value across a set of rows related to the current row, without collapsing the result set, using clauses like PARTITION BY and ORDER BY. Recursive CTEs allow you to define a query that references its own output, enabling traversal of trees or graphs. Query execution plans are visual or textual representations of the steps the engine will take, showing scans, joins, sorts, and memory usage. By reading a plan you can spot missing indexes, expensive sorts, or unnecessary row expansions, and then adjust your SQL or schema accordingly. These foundations give you the mental model to predict performance and to write correct, maintainable queries.

Window Functions

Window functions extend aggregate capabilities by operating over a sliding frame of rows defined relative to the current row. Unlike GROUP BY, they preserve the original row count, allowing calculations such as running totals, moving averages, and rank without collapsing data. The OVER clause specifies PARTITION BY to group rows and ORDER BY to define the frame order. Frame clauses like ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW control the exact subset. Trade-offs include increased memory usage for large partitions and potential sorting overhead; however, they often replace complex self-joins, making queries easier to read and maintain.

Worked reasoning: Consider a sales table with columns (region, sale_date, amount). To compute a cumulative sales total per region ordered by date, use: SELECT region, sale_date, amount, SUM(amount) OVER (PARTITION BY region ORDER BY sale_date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cum_total FROM sales; This returns each row with its running sum, preserving all original rows while avoiding a self-join.

Recursive CTEs

Recursive common table expressions let a query refer to its own output, enabling hierarchical traversals such as organizational charts or bill-of-materials explosions. A recursive CTE consists of an anchor member that provides the base rows and a recursive member that joins the CTE to other tables, typically using a UNION ALL. The recursion stops when the join yields no new rows, preventing infinite loops. Trade-offs involve careful termination conditions and potential exponential row growth; indexing the join columns and limiting recursion depth are common safeguards.

Worked reasoning: Given an employee table (emp_id, manager_id, name), retrieve the full management chain for employee 5: WITH RECURSIVE chain AS (SELECT emp_id, manager_id, name, 0 AS level FROM employee WHERE emp_id = 5 UNION ALL SELECT e.emp_id, e.manager_id, e.name, c.level+1 FROM employee e JOIN chain c ON e.emp_id = c.manager_id) SELECT * FROM chain ORDER BY level; This yields each manager up to the CEO, with a level indicating distance from the original employee.

Query Execution Plans

An execution plan is the database's roadmap for fulfilling a query. It lists operators such as Table Scan, Index Seek, Hash Join, and Sort, along with estimated row counts, costs, and memory usage. The plan is generated by the optimizer, which chooses the lowest-cost path based on statistics. Reading a plan helps you identify bottlenecks: a full table scan on a large table suggests a missing index; a costly Sort may be avoided with pre-sorted indexes; a high-cost Hash Join could be replaced by a Merge Join if inputs are already ordered. Trade-offs involve balancing index maintenance overhead against query speed, and sometimes accepting a higher CPU cost to reduce I/O.

Worked reasoning: For SELECT * FROM orders WHERE customer_id=42, inspect estimated versus actual rows in EXPLAIN ANALYZE. If one customer owns 50 of a million rows, a suitable index on customer_id can avoid reading the whole table. If the predicate returns most rows, a sequential scan may be the better plan. Compare execution time and buffer reads before and after the index; do not interpret optimizer cost units as milliseconds. Refresh stale statistics before forcing a different plan.

Worked examples

Worked example

```
import sqlite3
conn = sqlite3.connect(':memory:')
c = conn.cursor()
c.execute('CREATE TABLE sales(region TEXT, sale_date TEXT, amount INT)')
c.executemany('INSERT INTO sales VALUES (?,?,?)', [
    ('East','2023-01-01',100),
    ('East','2023-01-02',150),
    ('West','2023-01-01',200),
    ('East','2023-01-03',120)
])
query = '''
SELECT region, sale_date, amount,
       SUM(amount) OVER (PARTITION BY region ORDER BY sale_date
                         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cum
FROM sales ORDER BY region, sale_date;
'''
for row in c.execute(query):
    print(row)
conn.close()
```

The script creates a tiny sales table, inserts four rows, and runs a window function that computes a cumulative total per region. The output shows each original row plus a new column 'cum' with the running sum: ('East','2023-01-01',100,100), ('East','2023-01-02',150,250), ('East','2023-01-03',120,370), ('West','2023-01-01',200,200). Because the window is partitioned by region, the West row starts its own total at 200. This example demonstrates that no self-join is needed, but note that SQLite must materialize the partition, which can increase memory usage for large partitions.

Applied example

```
WITH RECURSIVE nums(n) AS (
  SELECT 1
  UNION ALL
  SELECT n+1 FROM nums WHERE n < 5
)
SELECT n, n*n AS square FROM nums;
```

The recursive CTE generates numbers 1 through 5 without a physical numbers table. The anchor SELECT 1 starts the recursion; the recursive member adds 1 until the condition n < 5 fails. The final SELECT computes each number's square. This pattern is useful for generating date ranges, pagination offsets, or test data. In production, ensure a LIMIT or MAXRECURSION setting to avoid runaway loops, and consider indexing the generated column if it will be joined later.

Process flow

1. 1. Identify the business question and decide whether aggregation, ranking, or hierarchy is needed.
2. 2. Choose a window function if you need row-wise calculations that preserve the result set.
3. 3. If the data is hierarchical, design a recursive CTE with a clear anchor and termination condition.
4. 4. Write the SQL query, adding PARTITION BY, ORDER BY, and frame clauses as required.
5. 5. Execute EXPLAIN (or EXPLAIN ANALYZE) to view the execution plan and spot scans or sorts.
6. 6. Add or adjust indexes based on the plan, then re-run the query to verify cost reduction.
7. 7. Validate results against a small hand-calculated dataset to ensure correctness.

Interview and exam checkpoints

Explain how a window function differs from a GROUP BY aggregation.

A window function computes a value across a set of rows related to the current row while keeping each input row in the output. GROUP BY collapses rows into a single result per group, losing the original row granularity. Window functions use the OVER clause with optional PARTITION BY and ORDER BY to define the frame, allowing calculations like running totals, ranks, or moving averages without a separate join. This makes them more expressive for analytical queries, though they may require extra sorting or memory for large partitions.

What are the termination conditions for a recursive CTE and why are they important?

A recursive CTE stops when the recursive member produces no new rows, which is the natural termination condition. Additionally, many databases enforce a maximum recursion depth (e.g., 100) to prevent infinite loops. Proper termination is crucial because without it the query could run indefinitely, consuming CPU and memory. Designing the anchor and recursive parts to converge - such as moving up a manager hierarchy until a NULL manager_id is reached - ensures predictable execution and protects the system.

How can you use an execution plan to decide whether to add an index?

When you run EXPLAIN, look for operators like TABLE SCAN or SORT that have high estimated costs. If a query filters on a column but the plan shows a full scan, adding an index on that column can turn the scan into an INDEX SEEK, reducing I/O. Similarly, if a Sort appears before a join, a composite index matching the ORDER BY can eliminate the sort. However, you must weigh the write overhead of maintaining the index against the read performance gains.

Describe a scenario where a window function could be replaced by a self-join, and discuss trade-offs.

To compute a running total, you could self-join the table on date <= current date and sum the amounts. This works but typically requires a GROUP BY on the joined rows, leading to O(n^2) row combinations and higher memory usage. A window function achieves the same result with a single pass and internal sorting, offering better performance and clearer syntax. The trade-off is that some older databases lack window support, forcing the self-join approach.

Summary

Advanced SQL equips you to answer analytical questions efficiently. Window functions let you compute aggregates across a moving frame while preserving rows, ideal for rankings and cumulative metrics. Recursive CTEs provide a declarative way to walk hierarchical data without procedural code, but require careful termination logic. Execution plans reveal how the optimizer will execute your query, guiding index creation and highlighting costly operations. By mastering these tools, you can write concise, high-performance queries, diagnose bottlenecks, and design schemas that support complex analytical workloads.

Practice questions

1. Given a table sales(product_id INT, sale_date TEXT, qty INT) with rows (1,'2023-01-01',10), (1,'2023-01-02',15), (2,'2023-01-01',5), compute the cumulative quantity per product using a window function.

2. Write a recursive CTE to list all ancestors of employee 4 given employee(emp_id, manager_id) rows (1,NULL),(2,1),(3,1),(4,2). Show the output.

3. A query plan shows a full table scan on orders (1,000,000 rows) followed by a sort on order_date. Which index would most likely eliminate both operations?

4. Explain why a window function with ROWS BETWEEN 2 PRECEDING AND CURRENT ROW might produce different results than using AVG(amount) GROUP BY with a HAVING clause.

5. Implement in Python a function that executes the recursive CTE from the previous practice question and returns the ancestor list as a Python list.

6. Design trade-off: You need to support both fast point lookups by order_id and frequent queries that compute a running total of order amounts per customer. Which indexing strategy would you choose and why?

Practice solutions

1. SQL: SELECT product_id, sale_date, qty,
SUM(qty) OVER (PARTITION BY product_id ORDER BY sale_date
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cum_qty
FROM sales ORDER BY product_id, sale_date;
Result rows: (1,'2023-01-01',10,10), (1,'2023-01-02',15,25), (2,'2023-01-01',5,5).

2. WITH RECURSIVE anc AS (
SELECT emp_id, manager_id FROM employee WHERE emp_id = 4
UNION ALL
SELECT e.emp_id, e.manager_id FROM employee e JOIN anc a ON e.emp_id = a.manager_id
)
SELECT emp_id FROM anc WHERE emp_id <> 4;
Output: 2, 1 (ancestors ordered from direct manager upward).

3. Create a composite index on orders(order_date). The index can satisfy the ORDER BY, allowing the optimizer to use an INDEX SEEK that returns rows already sorted, removing both the table scan and the explicit sort.

4. The window frame limits the calculation to the current row and the two rows before it, producing a moving average that changes for each row. GROUP BY aggregates over the entire partition, yielding a single average per group. HAVING filters groups after aggregation, not per row, so the semantics differ entirely.

5. import sqlite3

```
def get_ancestors(emp_id):
    conn = sqlite3.connect(':memory:')
    cur = conn.cursor()
    cur.execute('CREATE TABLE employee(emp_id INT, manager_id INT)')
    cur.executemany('INSERT INTO employee VALUES (?,?)', [(1,None),(2,1),(3,1),(4,2)])
    query = '''
    WITH RECURSIVE anc AS (
      SELECT emp_id, manager_id FROM employee WHERE emp_id = ?
      UNION ALL
      SELECT e.emp_id, e.manager_id FROM employee e JOIN anc a ON e.emp_id = a.manager_id
    )
    SELECT emp_id FROM anc WHERE emp_id <> ?;
    '''
    cur.execute(query, (emp_id, emp_id))
    result = [row[0] for row in cur.fetchall()]
    conn.close()
```

```
    return result
# get_ancestors(4) returns [2, 1]
```

6. Create a primary key index on order_id for O(log n) point lookups. Additionally, add a composite index on (customer_id, order_date, amount). This index supports the ORDER BY needed for the window function, allowing the database to read rows already sorted by date per customer, eliminating a separate sort. The trade-off is extra storage and write overhead for maintaining the composite index, but it yields fast point lookups and efficient running-total calculations.

Reference textbooks

Learning SQL - Alan Beaulieu

SQL Performance Explained - Markus Winand

Primary documentation

https://www.postgresql.org/docs/current/tutorial.html

Content version: learning-v1 | CareerIQ original AI-assisted educational notes

6. Supervised Learning Algorithms

Prerequisites: Linear Algebra, Probability & Statistics, NumPy & Pandas Processing

Supervised learning algorithms form the backbone of many real-world predictive systems, from credit scoring to image classification. They learn a mapping from input features to a target label by exploiting labeled examples. Understanding when to use a particular algorithm, how it builds its model, and what assumptions it makes is essential for any data scientist. This chapter introduces the fundamental ideas behind three widely used families - gradient boosted trees, regularization techniques for linear models, and support vector machines - starting from the mathematics that underpins them and progressing to practical implementation tips. By the end, a beginner will be able to select, train, and evaluate these models on small datasets, recognize common pitfalls, and discuss their trade-offs in an interview setting.

Foundations

Supervised learning begins with a dataset D = {(x_i, y_i)}_{i=1}^n where each x_i is a vector of p features and y_i is the target. The goal is to find a function f(x;θ) parameterized by θ that minimizes a loss L(y, f(x)). For regression, L is often the squared error (y - f(x))^2; for classification, the logistic loss or hinge loss is common. Linear models assume f(x)=w^T x + b, leading to a convex optimization problem that can be solved analytically or with gradient descent. Regularization adds a penalty term R(w) to the objective, controlling model complexity and preventing over-fitting. Tree-based methods, in contrast, partition the feature space recursively, fitting simple constant values in each leaf. Gradient boosting builds an ensemble of such trees by iteratively fitting each new tree to the residuals of the current ensemble, effectively performing gradient descent in function space. Support vector machines construct a separating hyperplane with maximal margin; by mapping inputs into a higher-dimensional space with a kernel, they can handle non-linear patterns while still solving a convex quadratic program. All three approaches share the need for careful preprocessing, hyper-parameter tuning, and validation to achieve reliable performance.

Gradient Boosted Trees (XGBoost)

Gradient boosted trees combine many shallow decision trees to produce a strong predictor. Each tree is fit to the negative gradient of the loss with respect to the current model predictions, which is why the method is called gradient boosting. The algorithm proceeds in stages: start with a constant prediction (often the mean of y), compute residuals r_i = -∂L/∂f(x_i), fit a regression tree to r_i, and update the model f_new(x) = f_old(x) + η * tree(x) where η is the learning rate. Regularization appears both in the tree structure (max depth, min child weight) and in the objective (L2 penalty on leaf weights). XGBoost adds second-order Taylor approximation of the loss, enabling efficient computation of optimal leaf values and providing built-in handling of missing values. Trade-offs include higher predictive power versus longer training time and greater sensitivity to noisy features if depth is not limited.

Worked reasoning: Consider a tiny dataset with a single feature x and target y: (1,3), (2,5), (3,7), (4,9). The initial prediction is the mean y = 6. The residuals are [-3, -1, 1, 3]. Fit a depth-1 tree that splits at x=2.5, assigning leaf values -2 for the left side and +2 for the right side (average residuals). With learning rate η=0.5, the updated predictions become 6 + 0.5*(-2) = 5 for x≤2.5 and 6 + 0.5*(2) = 7 for x>2.5. A second iteration repeats the process on new residuals, gradually shaping a piecewise linear approximation of the true relationship y=2x+1.

Regularization (L1/L2)

Regularization adds a penalty to the loss function to discourage overly complex models. L2 regularization (ridge) adds λ||w||_2^2, shrinking all coefficients toward zero but never exactly zero, which improves numerical stability and reduces variance. L1 regularization (lasso) adds λ||w||_1, encouraging sparsity by driving some coefficients exactly to zero, thus performing feature selection. The combined elastic-net penalty λ1||w||_1 + λ2||w||_2^2 balances both effects. In linear regression, the closed-form ridge solution is w = (X^T X + λI)^{-1} X^T y, while lasso requires iterative solvers such as coordinate descent. Trade-offs: L2 is easier to compute and works well when many small effects exist; L1 is useful when only a few features are truly predictive but can be unstable if features are highly correlated.

Worked reasoning: Take a 3-feature dataset: X = [[1,2,3],[1,0,1],[0,1,1]], y = [6,4,5]. Without regularization, the ordinary least squares solution solves (X^T X)w = X^T y. X^T X = [[2,2,4],[2,5,5],[4,5,11]] and X^T y = [15,14,27]. Solving gives w ≈ [1,1,1]. With L2 λ=1, compute (X^T X + I) = [[3,2,4],[2,6,5],[4,5,12]]. Inverting yields w_ridge ≈ [0.78,0.92,0.84]. The coefficients shrink toward zero, reducing sensitivity to noise.

Support Vector Machines

Support vector machines (SVM) aim to find the hyperplane that maximizes the margin between two classes. For linearly separable data, the primal problem minimizes ½||w||^2 subject to y_i (w^T x_i + b) ≥ 1. Introducing slack variables ξ_i and a penalty C yields the soft-margin formulation, allowing some misclassifications. The dual problem involves Lagrange multipliers α_i and depends only on inner products x_i·x_j, enabling the kernel trick: replace the inner product with a kernel function K(x_i,x_j) to implicitly map data into a high-dimensional space. Common kernels include polynomial K = (γ x_i·x_j + r)^d and radial basis function K = exp(-γ||x_i - x_j||^2). Only points with α_i > 0 (support vectors) influence the decision boundary. Trade-offs: SVMs provide strong theoretical guarantees and work well on medium-size datasets, but training scales poorly with n and requires careful kernel and C selection.

Worked reasoning: A binary dataset: x = [[0,0],[2,2],[2,0],[0,2]], y = [-1,-1,1,1]. The optimal separating line is x_1 = x_2. The margin is √2/2. The support vectors are (0,0) and (2,2) for class -1 and (2,0) and (0,2) for class +1. Solving the dual yields α = [0.5,0.5,0.5,0.5]. The decision function f(x) = Σ α_i y_i K(x_i,x) + b simplifies to f(x) = x_1 - x_2, correctly classifying all points.

Worked examples

Worked example

```
import numpy as np
X = np.array([[1],[2],[3],[4]])
y = np.array([3,5,7,9])
# initial constant prediction
f0 = np.mean(y)
res = y - f0
# simple depth-1 tree split at 2.5
left = res[X[:,0]<=2.5].mean()
right = res[X[:,0]>2.5].mean()
eta = 0.5
pred = f0 + eta*np.where(X[:,0]<=2.5, left, right)
print('pred:', pred)
```

The code implements the first two boosting iterations on a tiny regression problem. The initial prediction f0 = 6 is the mean of y. Residuals are computed, then a single split at x=2.5 yields average residuals -2 (left)

and +2 (right). With learning rate 0.5 the updated predictions become [5,5,7,7]. The output shows how gradient boosting incrementally improves the fit. Limitations include the use of only one tree depth and a very small dataset; real-world models require many trees, regularization, and validation to avoid over-fitting.

Applied example

```
def train_svm(X, y, C=1.0, gamma=0.5):
    # Pseudocode for soft-margin RBF SVM
    K = np.exp(-gamma * np.sum((X[:,None]-X[None,:])**2, axis=2))
    # Solve dual: maximize sum(alpha) - 0.5*alpha^T (YKY) alpha
    # subject to 0 <= alpha_i <= C and sum(alpha_i y_i)=0
    alpha = quadratic_program_solver(K, y, C)
    sv = alpha > 1e-6
    w = np.sum(alpha[sv][:,None] * y[sv][:,None] * X[sv], axis=0)
    b = np.mean(y[sv] - K[sv][:,sv] @ (alpha[sv] * y[sv]))
    return w, b
# Note: quadratic_program_solver is a placeholder for a QP library.
```

The pseudocode outlines the steps to train a soft-margin SVM with an RBF kernel. First, the kernel matrix K is built using the Gaussian formula. The dual optimization problem is then solved for the Lagrange multipliers alpha, respecting box constraints and the equality constraint. Support vectors are identified where alpha is non-zero. The weight vector w in the implicit feature space is reconstructed as a weighted sum of support vectors, and the bias b is computed by averaging over support vectors. In practice, a library such as cvxopt or scikit-learn handles the quadratic programming. The example highlights the importance of choosing C and gamma: large C reduces regularization, while large gamma makes the kernel narrow, both affecting margin and over-fitting.

Process flow

1. 1. Load and clean the data, ensuring numeric types and handling missing values.
2. 2. Split the dataset into training and validation sets to estimate generalization error.
3. 3. Choose a model family (tree boosting, regularized linear, or SVM) based on data size and feature types.
4. 4. Define the loss function and add regularization terms if applicable; set hyper-parameters.
5. 5. Train the model: for boosting iterate tree fitting on residuals; for linear solve the regularized normal equations; for SVM solve the dual quadratic program.
6. 6. Evaluate on validation data using appropriate metrics (RMSE for regression, accuracy/F1 for classification).
7. 7. Perform hyper-parameter tuning (grid or Bayesian) and repeat steps 4-6 until performance stabilizes.

Interview and exam checkpoints

Explain how gradient boosting reduces bias and variance compared to a single decision tree.

A single decision tree often has high variance because small changes in the data can produce a very different structure; it also may be shallow, leading to bias. Gradient boosting builds an ensemble where each new tree corrects the residual errors of the current model, effectively performing functional gradient descent. By adding many weak learners, the bias is reduced as the ensemble can approximate complex functions. Simultaneously, regularization techniques such as learning rate, tree depth limits, and subsampling control variance, preventing the ensemble from over-fitting the training data.

What is the effect of L1 regularization on feature selection, and when might it be harmful?

L1 regularization adds a penalty proportional to the absolute value of coefficients, encouraging many weights to become exactly zero. This yields an implicit feature selection, which is useful when the true model is sparse. However, if features are highly correlated, L1 may arbitrarily keep one and discard others, leading to instability and loss of predictive information. In such cases, an elastic-net or pure L2 penalty may be preferable.

Describe the kernel trick and give an example of a kernel that maps data to an infinite-dimensional space.

The kernel trick replaces the inner product $x_i \cdot x_j$ in the dual SVM formulation with a kernel function $K(x_i,x_j)$ that computes the inner product in a transformed feature space without explicitly performing the transformation. The radial basis function kernel $K = \exp(-\gamma||x_i - x_j||^2)$ corresponds to mapping data into an infinite-dimensional Hilbert space of Gaussian basis functions. This allows the SVM to create highly non-linear decision boundaries while keeping the optimization problem dependent only on pairwise kernel evaluations.

How does the learning rate η influence the training of a gradient boosted model?

The learning rate scales the contribution of each newly fitted tree to the ensemble. A small η (e.g., 0.01) forces the model to take many incremental steps, often improving generalization because each tree makes only a modest correction, reducing the risk of over-fitting. Conversely, a large η (e.g., 0.5) speeds up convergence but can cause the model to fit noise quickly, leading to higher variance. Practitioners typically pair a small η with a larger number of trees.

Summary

Supervised learning algorithms translate labeled data into predictive functions by minimizing a loss while controlling model complexity. Gradient boosted trees iteratively fit shallow trees to residuals, offering high accuracy at the cost of training time and hyper-parameter sensitivity. Regularization, through L1 or L2 penalties, shrinks linear model coefficients to reduce variance; L1 also performs feature selection but can be unstable with correlated inputs. Support vector machines construct maximal-margin hyperplanes, leveraging kernels to handle non-linear patterns, yet they scale poorly with large datasets. Across all methods, proper data preprocessing, validation, and systematic hyper-parameter tuning are essential to balance bias, variance, and computational resources.

Practice questions

1. Given X=[[1,2],[2,1],[3,3]], y=[4,5,10], compute the ridge regression coefficients with λ=2 using the closed-form solution.

2. Fit a depth-1 decision stump to the residuals r=[-2,-2,2,2] of a 4-point dataset with feature x=[1,2,3,4]. What are the leaf predictions and the updated model after one boosting step with η=0.5?

3. Implement Lasso regression using coordinate descent for a 2-feature dataset X=[[1,0],[0,1],[1,1]], y=[1,2,3] with λ=0.5. Show one full cycle of updates.

4. Design a binary classifier for a dataset with 10,000 samples and 500 features where most features are sparse. Choose between a linear SVM and gradient boosted trees and justify the trade-off.

5. Compute the squared approximation error and the L2 norm error for approximating vector v=[3,4] by u=[2,5].

6. Using scikit-learn, write a short Python snippet to train an XGBoost classifier with max_depth=3 and evaluate accuracy on a hold-out set.

Practice solutions

1. First compute X^T X = [[1+4+9, 2+2+9],[2+2+9,4+1+9]] = [[14,13],[13,14]]. Add λI: [[16,13],[13,16]]. Compute the inverse: determinant = 16*16-13*13 = 256-169 = 87. Inverse = (1/87)*[[16,-13],[-13,16]]. Compute X^T y = [1*4+2*5+3*10, 2*4+1*5+3*10] = [4+10+30, 8+5+30] = [44,43]. Multiply: w = (1/87)*[[16,-13],[-13,16]] @ [44,43] = (1/87)*[16*44-13*43, -13*44+16*43] = (1/87)*[704-559, -572+688] = (1/87)*[145,116] ≈ [1.666,1.333]. Thus the ridge coefficients are w1≈1.67, w2≈1.33.

2. A depth-1 stump chooses a split that separates negative from positive residuals. Splitting at x=2.5 yields left leaf residuals [-2,-2] with mean -2, right leaf residuals [2,2] with mean 2. With learning rate η=0.5, leaf contributions are -1 and +1 respectively. If the current model prediction is f0 (e.g., the mean of y), the updated prediction becomes f1 = f0 + leaf_contrib. For example, if f0=6, predictions become 5 for x≤2.5 and 7 for x>2.5.

3. Initialize w=[0,0]. Update w1: residual = y - X[:,1]*w2 = [1,2,3] - [0,1,1]*0 = [1,2,3]. Compute rho = X[:,0]^T residual = 1*1+0*2+1*3 = 4. Apply soft-threshold: w1 = sign(rho)*max(|rho|-λ,0)/ (X[:,0]^T X[:,0]) = 1*max(4-0.5,0)/2 = 3.5/2 = 1.75. Update w2: residual = y - X[:,0]*w1 = [1,2,3] - [1,0,1]*1.75 = [-0.75, 2, 1.25]. rho = X[:,1]^T residual = 0*(-0.75)+1*2+1*1.25 = 3.25. w2 = sign(3.25)*max(3.25-0.5,0)/2 = 2.75/2 = 1.375. After one cycle, w≈[1.75,1.38].

4. A linear SVM with an L1 penalty is preferable for high-dimensional sparse data because it operates on inner products and can exploit sparsity, leading to fast training and a model that uses only a subset of features. Gradient boosted trees would need to examine many dense splits, incurring high memory and time costs, and may over-fit due to the large feature space. The trade-off is that trees can capture non-linear interactions, but with 500 sparse features the linear decision boundary is often sufficient and far more scalable.

5. Squared approximation error = Σ (v_i - u_i)^2 = (3-2)^2 + (4-5)^2 = 1^2 + (-1)^2 = 2. L2 norm error = sqrt(2) ≈ 1.414. The squared error is useful in loss functions; the norm error gives the Euclidean distance.

6. import numpy as np
from sklearn.model_selection import train_test_split
from xgboost import XGBClassifier
X = np.random.randn(200,5)
y = (X[:,0] + X[:,1] > 0).astype(int)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = XGBClassifier(max_depth=3, n_estimators=50, use_label_encoder=False, eval_metric='logloss')
model.fit(X_train, y_train)
acc = model.score(X_test, y_test)
print('Accuracy:', acc)

Reference textbooks

Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow - Aurelien Geron

Pattern Recognition and Machine Learning - Christopher M. Bishop

The Elements of Statistical Learning - Trevor Hastie, Robert Tibshirani and Jerome Friedman

Primary documentation

https://scikit-learn.org/stable/supervised_learning.html

Content version: learning-v1 | CareerIQ original AI-assisted educational notes

7. Model Evaluation & Validation

Prerequisites: Supervised Learning Algorithms, Probability & Statistics

Model evaluation and validation are the final checkpoints that tell us whether a machine-learning system will succeed in the real world. They answer the questions of how well a model predicts, whether its performance is stable across unseen data, and whether the metrics we report truly reflect the business goal. In supervised learning, evaluation begins after a model is trained on a labeled dataset; we then measure its predictions on data that were not used for fitting. Validation goes a step further: it designs experiments - such as cross-validation or hold-out splits - to estimate how the model will behave on future data and to guard against subtle errors like data leakage. For large language models (LLMs), the same principles apply, but the metrics shift toward generation quality, factuality, and alignment. Understanding these foundations lets a practitioner choose the right metric, avoid common pitfalls, and communicate results with confidence, which is essential for both production deployments and technical interviews.

Foundations

The core of model evaluation rests on two ideas: a loss function that quantifies error on a single prediction, and an aggregation method that summarizes error over many predictions. For classification, loss functions such as cross-entropy produce a probability for each class; we then derive metrics like accuracy, precision, recall, and the area under the ROC curve (AUC). Each metric emphasizes a different aspect of performance: accuracy treats all errors equally, precision focuses on false positives, recall on false negatives, and AUC measures the ability to rank positive instances higher than negatives across all thresholds. Validation strategies - simple hold-out, k-fold cross-validation, and nested cross-validation - provide unbiased estimates of these metrics by repeatedly training on subsets of data and testing on the complementary parts. The most dangerous mistake is data leakage, where information from the test split unintentionally influences the training process, inflating performance. For LLMs, evaluation expands to include BLEU, ROUGE, BERTScore, and newer alignment-oriented metrics such as TruthfulQA or GPT-Eval, each requiring careful construction of reference texts and human-in-the-loop verification. By mastering these concepts, a beginner can move from raw predictions to trustworthy, reproducible performance reports.

ROC-AUC & Precision-Recall

The Receiver Operating Characteristic (ROC) curve plots the true-positive rate (TPR) against the false-positive rate (FPR) as the decision threshold varies from 0 to 1. The area under this curve (AUC) is a threshold-independent measure of ranking quality: an AUC of 0.5 means random guessing, while 1.0 indicates perfect separation. Precision-Recall (PR) curves, on the other hand, plot precision versus recall and are more informative when the positive class is rare. The area under the PR curve (average precision) emphasizes the model's ability to retrieve the few positive cases without being swamped by false positives. Trade-offs arise because ROC-AUC can look optimistic on imbalanced data, whereas PR-AUC penalizes missed positives more heavily. Choosing between them depends on the cost of false alarms versus missed detections in the target application.

Worked reasoning: Consider a binary classifier that outputs scores [0.9, 0.8, 0.4, 0.3] for four instances with true labels [1, 0, 1, 0]. Sorting by score gives pairs (0.9,1), (0.8,0), (0.4,1), (0.3,0). The ROC points are (FPR,TPR): (0,0), (0,0.5), (0.5,0.5), (0.5,1), (1,1). The trapezoidal AUC = 0.75. For PR, precision at recall 0.5 is 1/(1+1)=0.5, at recall 1.0 precision = 2/4=0.5, giving average precision = 0.5. This shows ROC-AUC

looks better than PR-AUC because positives are scarce.

Cross-Validation & Leakage

Cross-validation (CV) splits the data into k folds; each fold is held out once as a test set while the model trains on the remaining k-1 folds. The average metric across folds estimates out-of-sample performance and reduces variance compared to a single hold-out split. Nested CV adds an inner loop for hyper-parameter tuning, preventing optimistic bias from selecting parameters on the same data used for evaluation. Leakage occurs when information from the test fold leaks into training, for example by scaling features with the global mean or by performing feature selection before the split. Such leakage inflates performance because the model indirectly sees the test distribution. Detecting leakage requires strict ordering: split first, then compute any statistics (means, variances, encodings) exclusively on the training portion.

Worked reasoning: Suppose we have 8 samples with a single numeric feature X = [1,2,3,4,5,6,7,8] and binary label Y = [0,0,1,0,1,1,0,1]. In 4-fold CV, fold 1 contains indices 0,1; training set is indices 2-7. If we compute the mean of X on the whole dataset (4.5) and subtract it from all samples before splitting, the test folds have already been centered using information from the held-out points, causing leakage. Correct practice: compute mean on training indices only, e.g., for fold 1 mean(X_train)= (3+4+5+6+7+8)/6 = 5.5, then center training and test separately.

LLM Benchmark Metrics

Large language models generate free-form text, so evaluation must capture both lexical similarity and semantic fidelity. Traditional n-gram metrics like BLEU count overlapping word sequences but ignore meaning; ROUGE improves recall of n-grams, useful for summarization. BERTScore embeds tokens with a pretrained transformer and computes cosine similarity, offering a semantic match. More recent alignment metrics - such as TruthfulQA score, GPT-Eval, or OpenAI’s Evals - measure factual correctness, helpfulness, and safety by comparing model outputs against curated answer keys or by prompting a second LLM to judge. These metrics often combine automatic scoring with human annotation to calibrate thresholds. Trade-offs involve computational cost (BERTScore requires GPU inference) versus interpretability (BLEU is easy to explain). Selecting a metric depends on the downstream task: for code generation, exact match is critical; for open-ended chat, safety and truthfulness dominate.

Worked reasoning: A prompt asks an LLM to answer: "What is the capital of France?" The model replies "Paris". Using BLEU, the reference is "Paris"; BLEU = 1.0. BERTScore also yields 1.0 because the embeddings match. If the model says "The capital is Paris, known for its art," BLEU drops to 0.67 due to extra tokens, but BERTScore remains near 0.99, reflecting semantic correctness. A TruthfulQA evaluator might assign a binary score of 1 for factual correctness, showing how different metrics capture distinct aspects.

Worked examples

Worked example

```
import numpy as np

y_true = np.array([1,0,1,0,1])
y_score = np.array([0.9,0.2,0.8,0.4,0.3])
# sort by score
i = np.argsort(-y_score)
TP = np.cumsum(y_true[i])
FP = np.cumsum(1-y_true[i])
TPR = TP/TP[-1]
FPR = FP/FP[-1]
auc = np.trapz(TPR, FPR)
print('ROC AUC =', round(auc,3))
```

The code computes ROC-AUC without using any library function. After sorting the predicted scores in descending order, it cumulatively counts true positives (TP) and false positives (FP). Dividing by the total number of positives and negatives yields the true-positive and false-positive rates. The trapezoidal rule (np.trapz) integrates TPR over FPR, giving an AUC of 0.833 for this tiny dataset. This manual approach reveals how each threshold contributes to the curve and why AUC is insensitive to the exact probability values, only their ranking. Limitations include the need for at least one positive and one negative instance; otherwise the denominator becomes zero.

Applied example

```
def k_fold_cv(model, X, y, k=5):
    n = len(y)
    idx = np.arange(n)
    np.random.shuffle(idx)
    folds = np.array_split(idx, k)
    scores = []
    for i in range(k):
        test = folds[i]
        train = np.hstack([f for j,f in enumerate(folds) if j!=i])
        # fit on training only
        model.fit(X[train], y[train])
        pred = model.predict_proba(X[test])[:,1]
        scores.append(roc_auc_score(y[test], pred))
    return np.mean(scores)
# usage: avg_auc = k_fold_cv(LogisticRegression(), X, y)
```

The pseudocode outlines a generic k-fold cross-validation loop that respects the no-leakage rule: the model is fit only on the training indices, and predictions are made on the untouched test fold. After each iteration, the ROC-AUC for that fold is recorded, and the final report is the mean across folds. This pattern can be extended to nested CV by adding an inner loop that searches hyper-parameters on a further split of the training data. The example assumes a scikit-learn-like API; in practice, one must also reset random seeds for reproducibility and handle class-imbalance by stratified splitting.

Process flow

1. 1. Split the raw dataset into a hold-out test set that will never be touched during model development.

2	2. Within the remaining data, apply k-fold cross-validation to obtain stable estimates of the chosen metric.
3	3. For each fold, compute predictions, then derive ROC-AUC and Precision-Recall curves to understand ranking behavior.
4	4. Aggregate the fold-wise metrics (mean and standard deviation) to report expected performance and its variability.
5	5. Validate that no preprocessing step (scaling, encoding, feature selection) used information from the test fold; redo if leakage is detected.
6	6. For LLMs, supplement numeric scores with human-rated alignment checks or automated truthfulness benchmarks.
7	7. Document the entire pipeline, including random seeds, library versions, and metric definitions, to ensure reproducibility.

Interview and exam checkpoints

What is the difference between ROC-AUC and PR-AUC, and when would you prefer one over the other?

ROC-AUC measures the ability of a classifier to rank positives above negatives across all thresholds, treating false positives and false negatives equally. PR-AUC focuses on precision versus recall, emphasizing performance on the positive class. When the dataset is highly imbalanced and the positive class is rare, PR-AUC is more informative because a high ROC-AUC can be achieved by correctly classifying the abundant negative class while still missing many positives. Conversely, if both classes are balanced and the cost of false positives equals false negatives, ROC-AUC is a convenient summary.

Explain how data leakage can occur during cross-validation and how to prevent it.

Leakage happens when information from the test fold influences the training process. A common scenario is computing feature scaling parameters (mean, std) on the whole dataset before splitting; the test fold then indirectly contributes to the scaling, inflating performance. To prevent leakage, always perform any data-dependent transformation - scaling, encoding, feature selection - inside the training loop of each CV fold, using only the training portion to compute statistics, and then apply those statistics to the test portion.

Describe a metric you would use to evaluate a large language model that generates answers to factual questions.

For factual QA, a combination of exact match (or BLEU for lexical overlap) and a truthfulness metric such as TruthfulQA or GPT-Eval is appropriate. Exact match captures whether the answer string matches a reference, while TruthfulQA uses a curated set of questions with verified answers and scores the model's response for factual correctness, often via a secondary LLM or human annotators. This dual approach balances surface similarity with semantic correctness.

How does nested cross-validation differ from ordinary k-fold CV, and why is it useful?

Nested CV adds an inner loop inside each outer training fold to tune hyper-parameters. The outer loop provides an unbiased estimate of generalization performance, while the inner loop selects the best model configuration without peeking at the outer test data. This prevents optimistic bias that arises when hyper-parameters are tuned on the same data used for final evaluation, leading to more reliable performance estimates, especially when the dataset is small.

Summary

Model evaluation and validation turn raw predictions into trustworthy performance claims. ROC-AUC and Precision-Recall curves quantify ranking quality, with PR-AUC being more sensitive to class imbalance. Cross-validation offers robust estimates, but only when preprocessing respects the train-test boundary to avoid leakage; nested CV further safeguards hyper-parameter tuning. For large language models, traditional n-gram metrics are complemented by semantic scores like BERTScore and alignment-focused benchmarks such as TruthfulQA. Selecting the right metric depends on the problem's cost structure and data distribution, while a disciplined validation pipeline - hold-out test set, proper CV, leakage checks, and reproducible documentation - ensures that reported numbers reflect true future performance.

Practice questions

1. Given true labels [1,0,1,0,0,1] and predicted scores [0.9,0.4,0.8,0.3,0.2,0.7], compute the ROC-AUC manually using the trapezoidal rule.

2. Implement a stratified 3-fold cross-validation loop that reports both mean ROC-AUC and mean PR-AUC for a binary classifier using scikit-learn. Use a synthetic dataset of 12 points with 4 positives.

3. A dataset has 10,000 samples with 1% positive class. You obtain a model with ROC-AUC = 0.92 but PR-AUC = 0.15. Explain why the PR-AUC is low and what actions you could take to improve it.

4. Design a validation strategy for a time-series forecasting model that must avoid future data leakage. List the steps and justify each.

5. You are evaluating two LLMs on a summarization task. Model A gets BLEU = 0.45, ROUGE-L = 0.52, BERTScore = 0.78. Model B gets BLEU = 0.48, ROUGE-L = 0.49, BERTScore = 0.81. Which model would you recommend and why?

6. Explain a scenario where using only accuracy as an evaluation metric would be misleading, and propose a better metric.

Practice solutions

1. Sort by score descending: indices [0,2,5,1,3,4] -> pairs (0.9,1),(0.8,1),(0.7,1),(0.4,0),(0.3,0),(0.2,0). TP cumulative = [1,2,3,3,3,3]; FP cumulative = [0,0,0,1,2,3]. Total positives = 3, negatives = 3. TPR = TP/3 = [0.333,0.667,1,1,1,1]; FPR = FP/3 = [0,0,0,0.333,0.667,1]. Points: (0,0.333),(0,0.667),(0,1),(0.333,1),(0.667,1),(1,1). Trapezoidal area = sum over segments: (0-0)*(0.333+0.667)/2=0, (0-0)*(0.667+1)/2=0, (0.333-0)*(1+1)/2=0.333, (0.667-0.333)*(1+1)/2=0.334, (1-0.667)*(1+1)/2=0.333. Total AUC ≈ 1.0. Because all positives rank above all negatives, AUC = 1.

2. import numpy as np
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import roc_auc_score, average_precision_score
from sklearn.linear_model import LogisticRegression

X = np.arange(12).reshape(-1,1)
y = np.array([1,0,1,0,0,1,0,0,1,0,0,0])
skf = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)
roc_scores = []
pr_scores = []
for train_idx, test_idx in skf.split(X, y):
    model = LogisticRegression(solver='liblinear')
    model.fit(X[train_idx], y[train_idx])
    prob = model.predict_proba(X[test_idx])[:,1]
    roc_scores.append(roc_auc_score(y[test_idx], prob))
    pr_scores.append(average_precision_score(y[test_idx], prob))
print('Mean ROC-AUC:', np.mean(roc_scores))
print('Mean PR-AUC:', np.mean(pr_scores))

Explanation: StratifiedKFold keeps the proportion of positives (4/12) in each fold. The loop fits the model only on training indices, computes probabilities on the held-out test indices, and records both ROC-AUC and average precision (PR-AUC). The final printed means give a stable estimate of performance across the three folds.

3. With only 100 positive examples, ROC-AUC can be high even if the model ranks many negatives above some positives, because the false-positive rate denominator is large (9,900). PR-AUC, however, directly measures precision at various recall levels; a precision of 0.15 means that for every 100 retrieved items, only 15 are true positives, indicating many false positives. To improve PR-AUC you can: (1) adjust the decision threshold to a higher value, trading recall for precision; (2) use class-weighting or focal loss to force the model to focus on the minority class; (3) collect more positive examples or apply oversampling techniques such as SMOTE; (4) engineer features that better separate the rare class. Monitoring both metrics ensures you do not chase a misleading ROC-AUC.

4. 1. Preserve chronological order: split the data into training, validation, and test windows based on timestamps, never shuffling. 2. Use a rolling-origin (walk-forward) evaluation: after training on an initial window, forecast the next horizon, then expand the training window forward and repeat. 3. Compute metrics (e.g., MAE, MAPE) on each forecast horizon and average them to obtain a stable estimate. 4. Ensure any feature engineering (e.g., lag creation, scaling) is computed only on data available up to the forecast point; for scaling, fit the scaler on the training window and apply it to the validation/test windows without refitting. 5. Reserve the final untouched test period for the ultimate performance report. This strategy respects causality, prevents leakage of future information, and mimics real-world deployment where only past data is available at prediction time.

5. Model B has higher BLEU and BERTScore, indicating better lexical overlap and semantic similarity, while Model A scores higher on ROUGE-L, which emphasizes longest common subsequence recall. If the business goal values factual and semantic fidelity more than exact phrasing, Model B is preferable because BERTScore captures meaning and BLEU shows slightly better n-gram precision. If recall of longer phrases is critical, Model A might be chosen. Overall, given the modest ROUGE difference and the larger gains in BLEU and BERTScore, Model B is the stronger candidate for most summarization

applications.

6. In a fraud detection dataset with 99% legitimate transactions and 1% fraud, a naive model that predicts every transaction as legitimate achieves 99% accuracy but catches zero fraud cases. Accuracy is misleading because it does not reflect the cost of missing the rare but critical positive class. A better metric is the F1-score, which balances precision and recall, or the PR-AUC, which directly measures performance on the minority class. Alternatively, a cost-sensitive metric that assigns higher penalty to false negatives can be used to align evaluation with business impact.

Reference textbooks

Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow - Aurelien Geron

Pattern Recognition and Machine Learning - Christopher M. Bishop

The Elements of Statistical Learning - Trevor Hastie, Robert Tibshirani and Jerome Friedman

Primary documentation

https://scikit-learn.org/stable/modules/model_evaluation.html

Content version: learning-v1 | CareerIQ original AI-assisted educational notes

8. Unsupervised & Dimensionality

Prerequisites: NumPy & Pandas Processing

Unsupervised learning and dimensionality reduction are core techniques for extracting structure from data without explicit labels. They help visualize high-dimensional datasets, compress information, and prepare features for downstream tasks. When data has many variables, distances become noisy and algorithms can suffer; reducing dimensions to the most informative axes reveals clusters, trends, and anomalies. This chapter introduces three foundational methods - Principal Component Analysis (PCA), K-Means and DBSCAN clustering, and the manifold-learning visualizers t-SNE and UMAP. You will learn the mathematics that underpins each algorithm, see step-by-step numeric examples, and practice coding them in Python. By the end you will know when to apply each technique, how to interpret results, and how to avoid common pitfalls such as over-compression, sensitivity to scale, and stochastic variability.

Foundations

All three methods start from a data matrix X of shape (n_samples, n_features). The first step is always to centre the data by subtracting the column means; many algorithms also require scaling to unit variance so that each feature contributes equally to distance calculations. PCA seeks orthogonal directions (principal components) that maximise variance. It solves the eigenvalue problem of the covariance matrix C = (1/(n-1)) X^T X, yielding eigenvectors (directions) and eigenvalues (explained variance). Selecting the top k eigenvectors forms a projection matrix W_k, and the reduced data Z = X W_k retains the most variance while discarding noise. K-Means partitions points into k clusters by iteratively assigning each point to the nearest centroid and recomputing centroids as the mean of assigned points; the objective is to minimise the sum of squared Euclidean distances (inertia). DBSCAN, by contrast, defines clusters as dense regions separated by low-density gaps. It uses two parameters: eps (neighbourhood radius) and minPts (minimum points to form a dense region). Points are labelled core, border, or noise, and clusters grow by density-reachability. t-SNE and UMAP are non-linear techniques that preserve local neighbourhood structure. t-SNE converts high-dimensional pairwise distances into conditional probabilities, then finds a low-dimensional embedding that matches these probabilities using gradient descent. UMAP builds a fuzzy topological graph from the data and optimises a low-dimensional layout that preserves both local and global structure. Both require careful tuning of perplexity (t-SNE) or n_neighbors/min_dist (UMAP) and are stochastic, so multiple runs may differ. Understanding these foundations - centering, scaling, distance metrics, eigen decomposition, and density concepts - prepares you to apply each method correctly and interpret its output.

PCA Decomposition

PCA transforms correlated variables into a set of linearly uncorrelated components ordered by variance. The covariance matrix captures pairwise linear relationships; its eigenvectors point in directions of maximal spread, while eigenvalues quantify how much variance each direction explains. By selecting the top k eigenvectors, we construct a projection that retains the majority of information while reducing dimensionality. Trade-offs include loss of interpretability - components are linear combinations of original features - and the assumption of linearity; non-linear structures are not captured. PCA is deterministic, fast for moderate sizes, and works well when variance aligns with signal, but it can be misled by outliers, which inflate variance along spurious directions.

Worked reasoning: Consider a 3-point dataset with two features: X = [[2,0],[0,2],[2,2]]. Centering gives [[1,-1],[-1,1],[1,1]]. Covariance C = (1/2) * [[(1^2+(-1)^2+1^2), (1*(-1)+(-1)*1+1*1)], [(same), ((-1)^2+1^2+1^2)]] = [[1.5,0.5],[0.5,1.5]]. Eigenvalues are 2.0 and 1.0; eigenvectors are [1,1]/sqrt2 and [1,-1]/sqrt2. Keeping the top component yields W = [0.707,0.707]. Projected values Z = X_centered * W = [0, -1.414, 1.414]. The first component captures most variance (2/3 of total).

K-Means & DBSCAN

K-Means clusters by minimising within-cluster sum of squares. It requires the number of clusters k a priori and assumes roughly spherical, equally sized clusters. The algorithm converges to a local optimum; different initial centroids can lead to different solutions, so multiple restarts are common. DBSCAN, in contrast, discovers clusters of arbitrary shape based on density. It does not need a preset number of clusters, but it needs eps and minPts. Points in low-density regions become noise, which is useful for outlier detection. However, DBSCAN struggles when clusters have varying densities, as a single eps cannot capture both. Both rely on Euclidean distance unless a different metric is supplied.

Worked reasoning: Take points: A(0,0), B(1,0), C(0,1), D(5,5), E(6,5), F(5,6). For K-Means with k=2, initial centroids at A and D lead to assignments: cluster1 = {A,B,C}, centroid1 = (0.33,0.33); cluster2 = {D,E,F}, centroid2 = (5.33,5.33). Inertia = sum of squared distances = (0.33^2+0.33^2)+(0.67^2+0.33^2)+(0.33^2+0.67^2)+(0.33^2+0.33^2)+(0.67^2+0.33^2)+(0.33^2+0.67^2)=2.0. For DBSCAN with eps=1.5, minPts=2, points A-C are dense and form one cluster, D-F form another; no noise points appear.

t-SNE & UMAP

Both methods aim to embed high-dimensional data into 2- or 3-dimensional space for visualization while preserving neighbourhood relationships. t-SNE converts pairwise distances into conditional probabilities using a Gaussian kernel in the high-dimensional space and a Student-t kernel in the low-dimensional space. It then minimises the Kullback-Leibler divergence between the two distributions via gradient descent. The perplexity parameter roughly controls the number of effective neighbours; too low yields many small clusters, too high merges distinct groups. UMAP builds a fuzzy simplicial set by estimating a local connectivity graph with a chosen number of neighbours, then optimises a low-dimensional layout using stochastic gradient descent on a cross-entropy loss. UMAP tends to preserve more global structure and runs faster than t-SNE, but both are stochastic, so results vary across runs. Careful seeding and parameter tuning are essential for reproducible visual insights.

Worked reasoning: Suppose we have four 3-D points: p1=(0,0,0), p2=(0,1,0), p3=(5,5,5), p4=(5,6,5). Pairwise Euclidean distances: d12=1, d34=1, d13≈8.66, d14≈8.66, d23≈8.66, d24≈8.66. With t-SNE perplexity=2, the algorithm will assign high probability to the close pairs (p1-p2, p3-p4) and low probability to all others. After optimisation, the 2-D embedding might place p1 and p2 near each other at (0,0) and (0,1), and p3 and p4 near each other at (5,5) and (5,6), preserving the two clusters while separating them.

Worked examples

Worked example

```
import numpy as np
X = np.array([[2,0],[0,2],[2,2]])
Xc = X - X.mean(axis=0)
C = np.cov(Xc, rowvar=False)
vals, vecs = np.linalg.eig(C)
W = vecs[:, np.argsort(vals)[-1]]
Z = Xc @ W
print('Projected:', Z)
```

The code follows the PCA steps described earlier: centering, computing the covariance matrix, eigen-decomposition, selecting the eigenvector with the largest eigenvalue, and projecting the data. The printed projected values are [0., -1.41421356, 1.41421356], matching the manual calculation. Because the dataset is tiny and perfectly aligned, the first component captures two-thirds of the total variance. The limitation is that PCA assumes linear relationships; if the points lay on a curved manifold, this projection would distort the structure.

Applied example

```
# Pseudocode for DBSCAN clustering
# Input: data matrix X, eps, minPts
# Output: cluster labels
for each point p in X:
    N = neighbours(p, eps)
    if |N| >= minPts:
        mark p as core
    else:
        mark p as noise
cluster_id = 0
for each core point p not yet assigned:
    cluster_id += 1
    expand_cluster(p, cluster_id)

function expand_cluster(p, cid):
    assign p to cid
    for each q in neighbours(p, eps):
        if q is noise: assign q to cid
        if q not assigned:
            assign q to cid
            if q is core: expand_cluster(q, cid)
```

The pseudocode captures the essence of DBSCAN: identify core points based on neighbourhood density, then grow clusters by recursively adding density-reachable points. Unlike K-Means, no centroid is computed, and the algorithm naturally labels outliers as noise. The main pitfalls are choosing eps too small (many points become noise) or too large (distinct clusters merge). Additionally, the algorithm's runtime can be O(n^2) without spatial indexing structures such as KD-trees.

Process flow

1. 1. Load raw data and inspect shape; handle missing values.
2. 2. Center each feature by subtracting its mean; optionally scale to unit variance.

3	3. Choose a dimensionality reduction method: PCA for linear variance, t-SNE/UMAP for non-linear structure.
4	4. Apply the method, obtain reduced coordinates, and evaluate explained variance (PCA) or visual separation (t-SNE/UMAP).
5	5. If clustering is needed, decide between K-Means (spherical clusters, known k) or DBSCAN (arbitrary shape, unknown k).
6	6. Run the clustering algorithm on the reduced data; compute silhouette score or inertia to assess quality.
7	7. Visualise results: scatter plot of reduced dimensions coloured by cluster label; interpret patterns and identify outliers.

Interview and exam checkpoints

Explain how PCA determines the number of components to retain.

PCA computes eigenvalues of the covariance matrix, each representing variance along its eigenvector. By sorting eigenvalues descending, we can calculate the cumulative explained variance ratio. The number of components is chosen when the cumulative ratio exceeds a threshold (e.g., 90%). This ensures that most of the data's variability is preserved while discarding noise. The choice balances compression against information loss; in practice we also examine scree plots for elbow points.

What are the main differences between K-Means and DBSCAN regarding cluster shape and noise handling?

K-Means assumes clusters are convex, roughly spherical, and of similar size because it minimises Euclidean distance to a centroid. It forces every point into a cluster, so outliers are assigned to the nearest centroid. DBSCAN defines clusters as dense regions separated by low-density gaps; it can discover arbitrarily shaped clusters and explicitly marks points that do not belong to any dense region as noise. Consequently, DBSCAN is more robust to outliers but struggles when clusters have varying densities.

Why is t-SNE considered stochastic, and how can you obtain reproducible visualisations?

t-SNE optimises a non-convex loss using gradient descent starting from random initial positions. Different random seeds lead to different local minima, producing varied embeddings. To obtain reproducible results, set the random_state (or seed) parameter, use the same perplexity and learning rate, and run the algorithm multiple times to verify stability. Even with a fixed seed, the method may still be sensitive to data preprocessing.

Describe a scenario where using PCA before K-Means improves clustering performance.

When the original feature space is high-dimensional and contains correlated or noisy dimensions, Euclidean distances become less meaningful (the curse of dimensionality). Applying PCA reduces dimensionality to the most informative axes, decorrelates features, and removes noise. K-Means then operates on a compact, variance-rich space, leading to tighter clusters, lower inertia, and more stable centroids. For example, clustering image patches with 100-pixel features benefits from reducing to 10 principal components before K-Means.

Summary

Unsupervised learning provides tools to discover hidden structure without labels. PCA offers a deterministic, linear compression that preserves maximal variance and is ideal for noise reduction and preprocessing. K-Means and DBSCAN give complementary clustering strategies: K-Means excels with spherical, equally sized groups and requires a preset k, while DBSCAN discovers arbitrarily shaped clusters and isolates outliers without needing k. t-SNE and UMAP are powerful visualisation techniques that maintain local neighbourhoods; t-SNE emphasises local detail at the cost of global layout, whereas UMAP balances both and runs faster. Proper data centering, scaling, and parameter tuning are essential across all methods. By following a systematic workflow - cleaning, scaling, reducing, clustering, and visualising - practitioners can extract meaningful patterns, detect anomalies, and prepare features for downstream supervised tasks.

Practice questions

1. Given X = [[4,2],[2,4],[3,3]], compute the first principal component manually and report the projected values.

2. Run K-Means with k=2 on points (0,0), (1,0), (0,1), (5,5), (6,5), (5,6). Show one iteration of assignment and centroid update.

3. Using DBSCAN with eps=1.5 and minPts=2 on the same six points, identify core, border, and noise points.

4. Explain how changing the perplexity parameter in t-SNE from 5 to 50 would affect the embedding of a dataset with 500 points.

5. Design a pipeline that uses PCA to reduce dimensionality before applying DBSCAN on a high-dimensional text embedding (e.g., 300-D). List the steps and justify each choice.

6. Compare the trade-offs between using t-SNE and UMAP for visualising a dataset of 10,000 gene expression profiles.

Practice solutions

1. Center X: mean = [3,3]; Xc = [[1,-1],[-1,1],[0,0]]. Covariance C = (1/2) * [[(1^2+(-1)^2+0^2), (1*(-1)+(-1)*1+0*0)], same] = [[1, -1],[-1,1]]. Eigenvalues = 2 and 0; eigenvector for λ=2 is [1,-1]/sqrt2. Projection matrix W = [0.707,-0.707]. Projected Z = Xc @ W = [(1*0.707 + -1*-0.707)=1.414, (-1*0.707 + 1*-0.707)=-1.414, (0) =0]. So Z = [1.414, -1.414, 0].

2. Initial centroids: c1=(0,0), c2=(5,5). Assignment: points A(0,0),B(1,0),C(0,1) are closer to c1; D(5,5),E(6,5),F(5,6) closer to c2. New centroids: c1 = mean of A,B,C = ((0+1+0)/3, (0+0+1)/3) = (0.33,0.33). c2 = mean of D,E,F = ((5+6+5)/3, (5+5+6)/3) = (5.33,5.33). Inertia after this iteration = sum of squared distances = 2.0 (as computed earlier).

3. Neighbourhoods (radius 1.5): A has B and C within distance 1, so |N|=3 → core. B has A and C within distance 1, core. C has A and B, core. D has E and F within distance 1, core. E has D and F, core. F has D and E, core. No points are border or noise because every point meets minPts. Thus two clusters: {A,B,C} and {D,E,F}.

4. Perplexity roughly sets the effective number of neighbours each point considers. With perplexity=5, the algorithm focuses on very local structure, often producing many small, tightly packed clusters and possibly fragmenting larger groups. Raising perplexity to 50 expands the neighbourhood, encouraging the embedding to respect broader relationships; clusters may merge, and the global layout becomes smoother. However, too high a perplexity can blur distinct small clusters, making them indistinguishable. For 500 points, a perplexity between 30 and 50 is typical, balancing local detail and global coherence.

5. 1. Load the 300-D embeddings and optionally remove zero-variance dimensions. 2. Standardise each dimension to zero mean and unit variance to ensure Euclidean distances are meaningful. 3. Apply PCA and retain enough components to explain ~95% of variance (often ~50 components). This removes noise and reduces computational cost for DBSCAN, whose neighbour queries scale with dimensionality. 4. Choose eps based on the average distance in the reduced space (e.g., using a k-distance plot) and set minPts to a small integer like 5. 5. Run DBSCAN on the PCA-reduced data to discover dense clusters of semantically similar documents while labeling outliers. 6. Evaluate cluster quality with silhouette score and inspect representative documents. The PCA step decorrelates features and mitigates the curse of dimensionality, improving DBSCAN's density estimation.

6. t-SNE excels at preserving very local neighbourhoods, often revealing fine-grained sub-clusters, but it is computationally intensive (O(N^2)) and requires careful tuning of perplexity; results can vary between runs. UMAP scales better (approximately O(N log N)), runs faster on 10,000 points, and tends to preserve both local and some global structure, making it easier to see overall relationships between major cell types. However, UMAP may slightly blur the tightest clusters compared to t-SNE. For exploratory analysis where speed and reproducibility matter, UMAP is preferred; for detailed sub-population discovery, t-SNE may be advantageous despite higher cost.

Reference textbooks

Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow - Aurelien Geron

Pattern Recognition and Machine Learning - Christopher M. Bishop

The Elements of Statistical Learning - Trevor Hastie, Robert Tibshirani and Jerome Friedman

Primary documentation

https://scikit-learn.org/stable/unsupervised_learning.html

Content version: learning-v1 | CareerIQ original AI-assisted educational notes